

M2.988 – Bases de Dades Analítiques

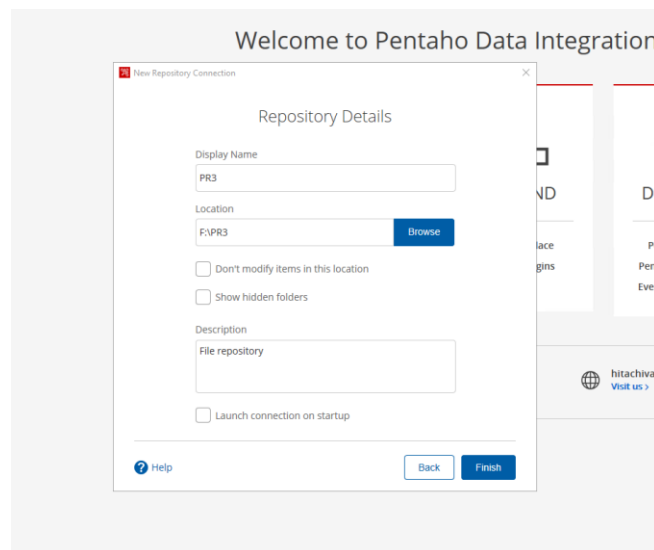
PR3: Sistemes de Bases de Dades Analítiques

Albert Carmona Hernández
16/12/2025

Exercici 1: Càrrega d'un fitxer a la base de dades

1.1 Configuració de Spoon

En primer lloc, entrem a *Spoon* i creem un repositori.

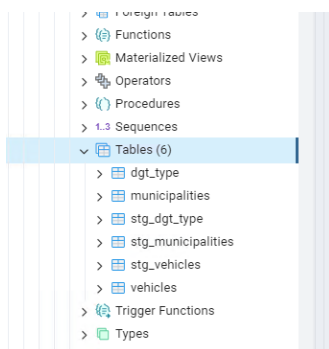


Afegim la variable *DIR_IN* per emmagatzemar la ruta de les fonts de les dades.

12	KETTLE_COMPATIBILITY_IGNORE_TIMEZONE	N
13	KETTLE_COMPATIBILITY_IMPORT_PATH_ADDITION_ON_VARIABLES	N
14	KETTLE_COMPATIBILITY_INVOKE_FILES_WITH_OR_WITHOUT_FILE_EXTENSION	Y
15	KETTLE_COMPATIBILITY_MERGE_ROWS_USE_REFERENCE_STREAM_WHEN_IDENTICAL	N
16	KETTLE_COMPATIBILITY_PUR_OLD_NAMING_MODE	N
17	KETTLE_COMPATIBILITY_TEXT_FILE_OUTPUT_APPEND_NO_HEADER	N
18	KETTLE_CORE_JOENTRIES_FILE	
19	KETTLE_CORE_STEPS_FILE	
20	KETTLE_DATA_REFINERY_HTTP_CLIENT_TIMEOUT	2000
21	KETTLE_DEFAULT_BIGNUMBER_FORMAT	
22	KETTLE_DEFAULT_DATE_FORMAT	
23	KETTLE_DEFAULT_INTEGER_FORMAT	
24	DIR_IN	F:\Fonts
25	KETTLE_DEFAULT_NUMBER_FORMAT	
26	KETTLE_DEFAULT_SERVLET_ENCODING	
27	KETTLE_DEFAULT_TIMESTAMP_FORMAT	
28	KETTLE_DISABLE_CONSOLE_LOGGING	N
29	KETTLE_DO_NOT_NORMALIZE_NULL_STRING_TO_EMPTY	N
30	KETTLE_DO_NOT_NORMALIZE_SPACES_ONLY_STRING_TO_EMPTY	N
31	KETTLE_EMPTY_STRING_DIFFERS_FROM_NULL	N
32	KETTLE_FAIL_ON_LOGGING_ERROR	N
33	KETTLE_FILE_OUTPUT_MAX_STREAM_COUNT	1024
34	KETTLE_FILE_OUTPUT_MAX_STREAM_LIFE	0
35	KETTLE_GLOBAL_LOG_VARIABLES_CLEAR_ON_EXPORT	false
..		..

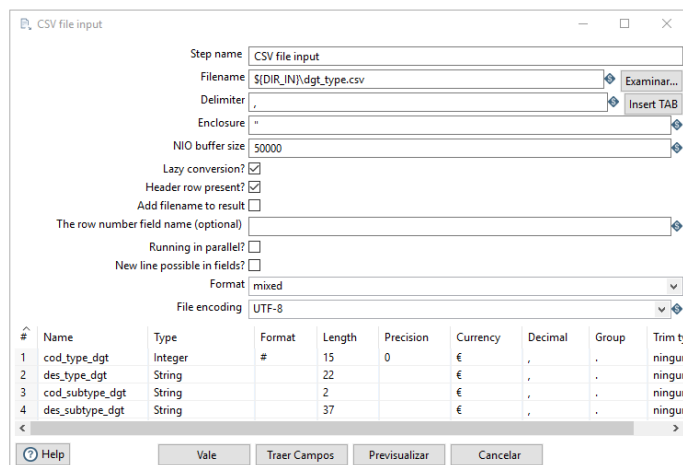
1.3 Creació de les taules a la base de dades a la capa de Staging

Creem les taules a la base de dades de *PostgreSQL* amb l'script proporcionat a l'enunciat. Una vegada creades, veiem com apareixen al menú de l'esquerra.



1.4 Càrrega dels fitxers

Per carregar les dades del *dgt_type.csv* es crea una transformació, *IN_DGT_TYPE*, a *Spoon* amb un primer pas *CSV file input*, on indiquem la ubicació de l'arxiu que volem importar (*dgt_type.csv*), especifiquem la codificació *UTF-8* i portem els camps. Aquest pas es pot veure a la següent imatge:



Si previsualitzem la taula obtenim el següent:

Examine preview data

Rows of step: CSV file input (112 rows)

#	cod_type_dgt	des_type_dgt	cod_subtype_dgt	des_subtype_dgt
1	1	CAMIONES	00	CAMIÓN
2	1	CAMIONES	01	CAMIÓN PLATAFORMA
3	1	CAMIONES	02	CAMIÓN CAJA
4	1	CAMIONES	03	CAMIÓN FURGÓN
5	1	CAMIONES	04	CAMIÓN BOTELLERO
6	1	CAMIONES	05	CAMIÓN CISTERNA
7	1	CAMIONES	06	CAMIÓN JAULA
8	1	CAMIONES	07	CAMIÓN FRIGORÍFICO
9	1	CAMIONES	08	CAMIÓN TALLER
10	1	CAMIONES	09	CAMIÓN PARA CANTERA
11	1	CAMIONES	10	CAMIÓN ARTICULADO
12	1	CAMIONES	11	CAMIÓN ARTICULADO PLATAFORMA
13	1	CAMIONES	12	CAMIÓN ARTICULADO CAJA
14	1	CAMIONES	13	CAMIÓN ARTICULADO FURGÓN
15	1	CAMIONES	14	CAMIÓN ARTICULADO BOTELLERO
16	1	CAMIONES	15	CAMIÓN ARTICULADO CISTERNA
17	1	CAMIONES	16	CAMIÓN ARTICULADO JAULA
18	1	CAMIONES	17	CAMIÓN ARTICULADO FRIGORÍFICO
19	1	CAMIONES	18	CAMIÓN ARTICULADO TALLER
20	1	CAMIONES	19	CAMIÓN ARTICULADO PARA CANTERA
21	2	FURGONETAS	20	FURGONETA
22	2	FURGONETAS	21	FURGONETA MIXTA
23	2	FURGONETAS	22	AMBULANCIA
24	2	FURGONETAS	23	COCHE FUNEBRE
25	2	FURGONETAS	24	CAMIONETA
26	2	FURGONETAS	25	TODO TERRENO
27	3	AUTOBUSES	30	AUTOBUS
28	3	AUTOBUSES	31	AUTOBUS ARTICULADO
29	3	AUTOBUSES	32	AUTOBUS MIXTO
30	3	AUTOBUSES	33	BIBLIOBUS
31	3	AUTOBUSES	34	AUTOBUS LABORATORIO
32	3	AUTOBUSES	35	AUTOBUS TALLER
33	3	AUTOBUSES	36	AUTOBUS SANITARIO

Cerrar Show Log

Connectem aquest pas amb un *Table output* per indicar la taula on volem carregar les dades (*stg_dgt_type*), configurant la connexió amb la base de dades de *PostgreSQL*, tal com es veu a la següent imatge:

Database Connection

General
Advanced
Options
Pooling
Clustering

Connection name: cn_stg

Connection type:

- Oracle
- Oracle RDB
- Palo MOLAP Server
- Pentaho Data Services
- PostgreSQL
- Redshift
- Remedy Action Request System
- SAP ERP System
- SQLite
- Snowflake
- SparkSQL
- Sybase
- SybaseIQ
- Teradata

Access:

- Native (JDBC)
- ODBC
- JNDI

Settings

Host Name: UCS1R1UOCSQL03

Database Name: SOURCE_acarmonaher

Port Number: 5432

Username: STUDENT_acarmonaher

Password: *****

Test Feature List Explore

OK Cancel

Una vegada configurada la connexió, indiquem la taula a la qual volem carregar les dades i seleccionem *Truncate table* i *Specify database fields* per indicar els camps que volem utilitzar.

Table output

Step name: Table output

Connection: cn_stg

Target schema: dbo

Target table: stg_dgt_type

Commit size: 1000

Truncate table: ☒

Ignore insert errors: ☐

Specify database fields: ☒

Main options: Database fields

Fields to insert:

#	Table field	Stream field
1	cod_type_dgt	cod_type_dgt
2	des_type_dgt	des_type_dgt
3	cod_subtype_dgt	cod_subtype_dgt
4	des_subtype_dgt	des_subtype_dgt

Buttons: Get fields, Enter field mapping

Buttons: Help, OK, Cancel, SQL

Executem la transformació i veiem que funciona correctament.



Execution Results

Logging Execution History Step Metrics Performance Graph Metrics Preview data

2025/12/09 12:58:56 - Spoon - Running transformation using the Kettle execution engine

2025/12/09 12:58:56 - Spoon - Transformation opened.

2025/12/09 12:58:56 - Spoon - Launching transformation [IN_DGT_TYPE]...

2025/12/09 12:58:56 - Spoon - Started the transformation execution.

2025/12/09 12:58:56 - IN_DGT_TYPE - Dispatching started for transformation [IN_DGT_TYPE]

2025/12/09 12:58:56 - Table output.0 - Connected to database [cn_stg] (commit=1000)

2025/12/09 12:58:56 - CSV file input.0 - Header row skipped in file 'F:\Fonts\dgt_type.csv'

2025/12/09 12:58:56 - CSV file input.0 - Finished processing (I=113, O=0, R=0, W=112, U=0, E=0)

2025/12/09 12:58:56 - Table output.0 - Finished processing (I=0, O=112, R=112, W=112, U=0, E=0)

2025/12/09 12:58:56 - Spoon - The transformation has finished!!

Una vegada executada, des de *PostgreSQL* podem veure que les dades s'han carregat correctament.

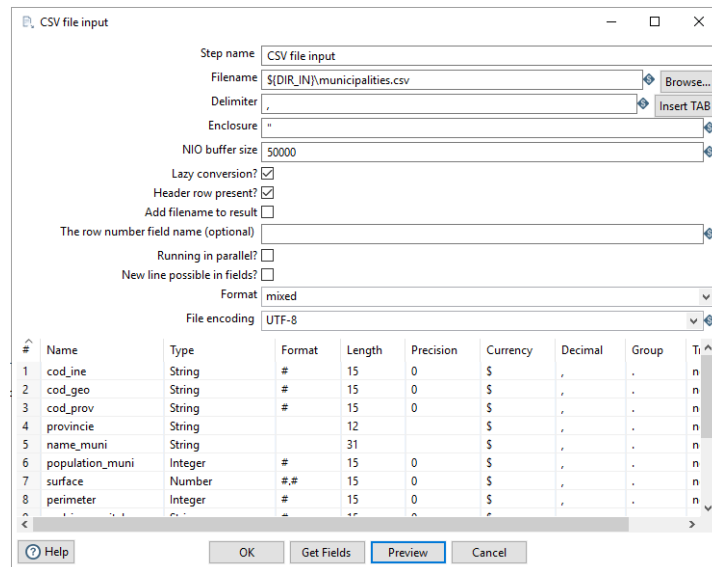
Query Query History

```
1 SELECT *
2 FROM dbo.stg_dgt_type
```

Data Output Messages Notifications

	cod_type_dgt character varying (1)	des_type_dgt character varying (64)	cod_subtype_dgt character varying (2)	des_subtype_dgt character varying (64)
1	1	CAMIONES	00	CAMIÓN
2	1	CAMIONES	01	CAMIÓN PLATAFORMA
3	1	CAMIONES	02	CAMIÓN CAJA
4	1	CAMIONES	03	CAMIÓN FURGÓN
5	1	CAMIONES	04	CAMIÓN BOTELLERO
6	1	CAMIONES	05	CAMIÓN CISTERNA
7	1	CAMIONES	06	CAMIÓN JAULA
8	1	CAMIONES	07	CAMIÓN FRIGORÍFICO

Per carregar les dades del *municipalities.csv* repetim el mateix procediment, important les dades amb un *CSV file input*, indicant l'arxiu que volem importar.



CSV file input

Step name: CSV file input

Filename: \$(DIR_IN)\municipalities.csv

Delimiter: ,

Enclosure: "

NIO buffer size: 50000

Lazy conversion? ☒

Header row present? ☒

Add filename to result? ☐

The row number field name (optional):

Running in parallel? ☐

New line possible in fields? ☐

Format: mixed

File encoding: UTF-8

#	Name	Type	Format	Length	Precision	Currency	Decimal	Group	Tr
1	cod_ine	String	#	15	0	\$	,	-	n
2	cod_geo	String	#	15	0	\$	,	-	n
3	cod_prov	String	#	15	0	\$	,	-	n
4	provincia	String		12		\$	,	-	n
5	name_muni	String		31		\$	,	-	n
6	population_muni	Integer	#	15	0	\$	,	-	n
7	surface	Number	#, #	15	0	\$	,	-	n
8	perimeter	Integer	#	15	0	\$	,	-	n

Help OK Get Fields Preview Cancel

En aquest cas, vaig veure que el tipus de les dades d'alguns camps, al indicar-se de manera automàtica, era incorrecte. Concretament, els camps *cod_ine*, *cod_geo*, *cod_prov* i *cod_ine_capital* s'importaven com a *integer*, i això feia que els zeros davant del número s'eliminaven (per exemple, un codi 08160, es convertia en 8160). Per solucionar-ho, he especificat el tipus de les dades d'aquests camps com *string*.

Connectem aquest pas a un *Table output*, utilitzant la connexió amb la base de dades de *PostgreSQL*, indicant la taula a la que es volen carregar les dades (*stg_municipalities*) i els camps corresponents. Finalment, executem la transformació.

Table output

Step name: Table output

Connection: cn_stg

Target schema: dbo

Target table: stg_municipalities

Commit size: 1000

Truncate table: ☒

Ignore insert errors: ☐

Specify database fields: ☒

Main options | Database fields

Fields to insert:

#	Table field	Stream field
1	cod_ine	cod_ine
2	cod_geo	cod_geo
3	cod_prov	cod_prov
4	provincia	provincia
5	name_muni	name_muni
6	population_muni	population_muni
7	surface	surface
8	perimeter	perimeter
9	cod_ine_capital	cod_ine_capital
10	capital	capital
11	population_capital	population_capital
12	longitude_etr89	longitude_etr89
13	latitude_etr89	latitude_etr89
14	altitude	altitude

Get fields

Enter field mapping

Help OK Cancel SQL



Execution Results

Logging Execution History Step Metrics Performance Graph Metrics Preview data

2025/12/09 13:34:25 - Spoon - Running transformation using the Kettle execution engine

2025/12/09 13:34:26 - Spoon - Transformation opened.

2025/12/09 13:34:26 - Spoon - Launching transformation [IN_MUNICIPALITIES]...

2025/12/09 13:34:26 - Spoon - Started the transformation execution.

2025/12/09 13:34:26 - IN_MUNICIPALITIES - Dispatching started for transformation [IN_MUNICIPALITIES]

2025/12/09 13:34:26 - Table output.0 - Connected to database [cn_stg] (commit=1000)

2025/12/09 13:34:26 - CSV file input.0 - Header row skipped in file 'F:\Fonts\municipalities.csv'

2025/12/09 13:34:26 - CSV file input.0 - Finished processing (I=8133, O=0, R=0, W=8132, U=0, E=0)

2025/12/09 13:34:27 - Table output.0 - Finished processing (I=0, O=8132, R=8132, W=8132, U=0, E=0)

2025/12/09 13:34:27 - Spoon - The transformation has finished!!

Comprovem que les dades s'han inserit correctament a la taula:

Query Query History

```

1 SELECT *
2 FROM dbo.stg_municipalities;
  
```

Data Output Messages Notifications

	cod_ine character varying (15)	cod_geo character varying (5)	cod_prov character varying (2)	provincia character varying (50)	name_muni character varying (50)
1	1001000000	1010	1	Araba/Álava	Alegria-Dulantzi
2	1002000000	1020	1	Araba/Álava	Amurrio
3	1003000000	1030	1	Araba/Álava	Aramaio
4	1004000000	1040	1	Araba/Álava	Artziniega
5	1006000000	1060	1	Araba/Álava	Armiñón
6	1008000000	1080	1	Araba/Álava	Arratzua-Ubarrundia
7	1009000000	1090	1	Araba/Álava	Asparrena
8	1010000000	1100	1	Araba/Álava	Ayala/Alara
9	1011000000	1110	1	Araba/Álava	Baños de Ebro/Mañueta

1.5 Exercicis a realitzar

Finalment, carreguem les dades de *vehicles.csv* amb una nova transformació anomenada *IN_VEHICLES*.

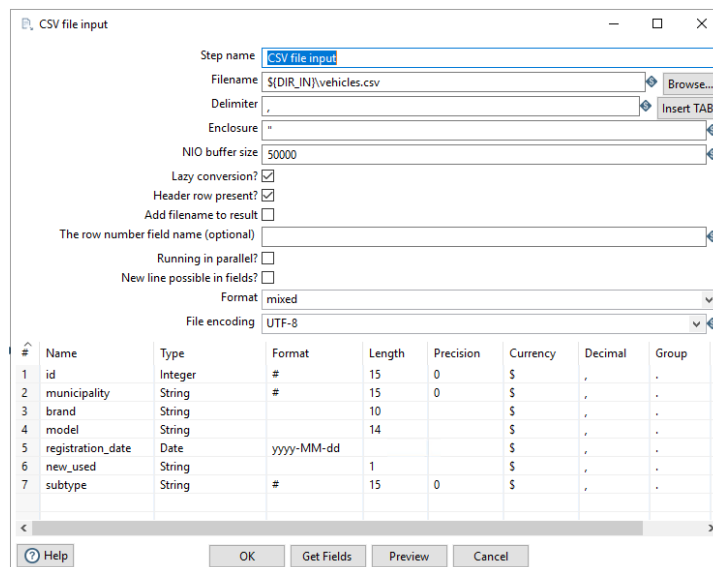
Primer es crea la taula intermèdia a *PostgreSQL* amb el següent script SQL:

```

DROP TABLE IF EXISTS dbo.stg_vehicles;
CREATE TABLE dbo.stg_vehicles (
    id integer NOT null,
    municipality varchar(5),
    brand varchar(25),
    model varchar(25),
    registration_date date,
    new_used boolean,
    subtype varchar(2)
);
  
```

En aquest cas, l'arxiu *csv* conté un camp *new_used* amb possibles valors “N” (New) i “U” (Used) que és un booleà simulat. A la taula creada a la base de dades es defineix el tipus de *new_used* com a *boolean*. Per poder carregar correctament les dades a la taula, necessitem que aquest camp sigui un *boolean*, ja que inicialment s'importa com a *string* al tractar-se de text. Si canviem el tipus al *file input*, no es fa correctament ja que no sap interpretar “N” i “U” com un *boolean*. Per solucionar-ho, entre el *file input* i el *table output*, he afegit dos passos amb l'objectiu de canviar els valors del camp *new_used* i convertir-los a *boolean*: *Value mapper* i *Select values*.

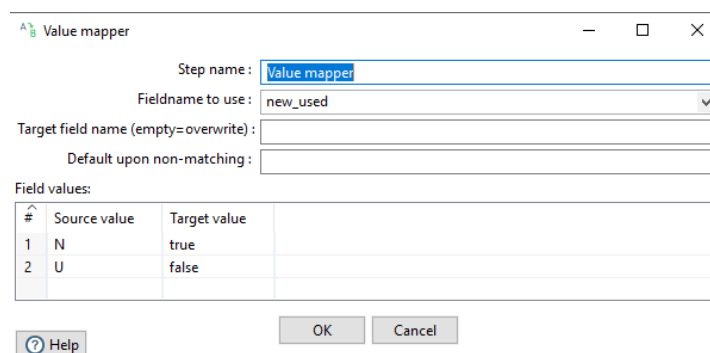
El *file input* ha quedat de la següent manera:



#	Name	Type	Format	Length	Precision	Currency	Decimal	Group
1	id	Integer	#	15	0	\$	,	-
2	municipality	String	#	15	0	\$	,	-
3	brand	String		10		\$	,	-
4	model	String		14		\$	,	-
5	registration_date	Date	yyyy-MM-dd			\$	,	-
6	new_used	String		1		\$	,	-
7	subtype	String	#	15	0	\$	,	-

Cal especificar que també he canviat el tipus del camp *municipality* a *string* pel mateix motiu que a la transformació anterior (evitar que canvi el valor si s'importa com un *integer*). Viem que a excepció de *new_used* (que s'ha de tractar a part), la resta de camps tenen el tipus corresponent als de la taula *stg_vehicles*.

Després del *file input* he utilitzat el *value mapper*, que permet canviar valors específics d'un camp d'un valor a un altre. En aquest cas he modificat els valors “N” Per “true” i “U” per “false”.



#	Source value	Target value
1	N	true
2	U	false

En aquest punt, *new_used* segueix sent un *string*. Per canviar-lo a *boolean* he utilitzat el *select values*, conservant tots els camps i modificant el de *new_used*.

Select values

Step name: Select values

Select & Alter Remove Meta-data

Fields:

#	Fieldname	Rename to	Length	Precision
1	id			
2	municipality			
3	brand			
4	model			
5	registration_date			
6	new_used			
7	subtype			

Get fields to select
Edit Mapping

Include unspecified fields, ordered by name ☐

Help OK Cancel

Select values

Step name: Select values

Select & Alter Remove Meta-data

Fields to alter the meta-data for:

#	Fieldname	Rename to	Type	Length	Precision	Binary to
1	new_used		Boolean			N

Get fields to change

Help OK Cancel

Finalment, es carreguen les dades a la taula de *PostgreSQL* amb un *table output*, de la mateixa manera que al les anteriors transformacions, establint la connexió amb la base de dades i indicant la taula i els camps.

Database Connection

General

Connection name: cn_stg

Connection type:

- Oracle
- Oracle RDB
- Palo MQLAP Server
- Pentaho Data Services
- PostgreSQL**
- Redshift
- Remedy Action Request System
- SAP ERP System
- SQLite
- Snowflake
- SparkSQL
- Sybase
- SybaseIQ
- Teradata
- UniVerse database
- Vertica
- Vertica 5+
- dBBase III, IV or 5

Settings:

Host Name: UCS1R1UOCSQL03

Database Name: SOURCE_acarmonaher

Port Number: 5432

Username: STUDENT_acarmonaher

Password: *****

Access:

- Native (ODBC)**
- ODBC
- JNDI

Test Feature List Explore

OK Cancel

Table output

Step name: Table output

Connection: cn_stg Edit... New... Wizard...

Target schema: dbo Browse...

Target table: stg_vehicled Browse...

Commit size: 1000

Truncate table ☒

Ignore insert errors ☐

Specify database fields ☒

Main options Database fields

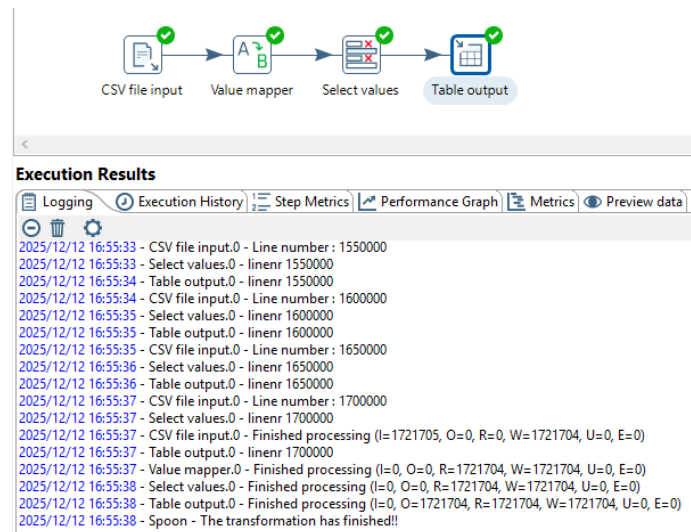
Fields to insert:

#	Table field	Stream field
1	id	id
2	municipality	municipality
3	brand	brand
4	model	model
5	registration...	registration_...
6	new_used	new_used
7	subtype	subtype

Get fields
Enter field mapping

Help OK Cancel SQL

Executem la transformació.



Comprovem amb una petita consulta que les dades s'han inserit correctament (els valors de *municipality* i de *new_used* són correctes).

Data Output								
Messages								
Notifications								
	id	municipality	brand	model	registration_date	new_used	subtype	
	integer	character varying (5)	character varying (25)	character varying (25)	date	boolean	character varying (2)	
13	303316	08156	FORD	TRANSIT CUSTOM	2020-03-01	true	20	
14	303317	08156	FORD	TRANSIT CUSTOM	2020-03-01	true	20	
15	303318	08156	FORD	TRANSIT CUSTOM	2020-03-01	true	20	
16	303319	08156	FORD	TRANSIT CUSTOM	2020-03-01	true	20	
17	303320	08156	FORD	TRANSIT CUSTOM	2020-03-01	true	20	
18	303321	08156	FORD	TRANSIT CUSTOM	2020-03-01	true	20	
19	303322	08156	FORD	TRANSIT CUSTOM	2020-03-01	true	20	
20	303323	08279	VOLKSWAGEN	TOURAN	2021-06-02	false	40	
21	303324	08279	VOLKSWAGEN	TOURAN	2021-06-02	false	40	
22	303325	08279	VOLKSWAGEN	TOURAN	2021-06-02	false	40	
23	303326	08279	VOLKSWAGEN	TOURAN	2021-06-02	false	40	
24	303327	08279	VOLKSWAGEN	TOURAN	2021-06-02	false	40	
25	303328	08279	VOLKSWAGEN	TOURAN	2021-06-02	false	40	
Total rows: 1000 of 1721704 Query complete 00:00:03.382								

Exercici 2: Anàlisi de la qualitat de les dades

▪ Taula *stg_dgt_type*

Per l'anàlisi de la qualitat de les dades de la taula primer realitzem una consulta per veure el nombre de registres emmagatzemats a la taula:

Query		Query History
<pre> 1 SELECT 2 COUNT(1) AS reg_dgt_type 3 FROM 4 dbo.stg_dgt_type; </pre>		
Data Output		Messages
		Notifications
	reg_dgt_type bigint	
1	112	

En segon lloc, executem l'script proporcionat a l'enunciat que, per cada columna de la taula, calcula el recompte de valors no nuls, el recompte de valors nuls, el percentatge de valors nuls el recompte de valors únics, la longitud mínima i la longitud màxima.

Query

Query History

```
1 WITH total_rows AS (
2 SELECT COUNT(*) AS total FROM dbo.stg_dgt_type
3 ),
4 col_analysis AS (
5     -- Anàlisi de la qualitat de les dades de cod_type_dgt
6 SELECT
7     'cod_type_dgt' AS columna,
8     COUNT(cod_type_dgt) AS no_nulos,
9     COUNT(distinct cod_type_dgt) AS valores_unicos,
10    min(length(cod_type_dgt)) as min_length,
11    max(length(cod_type_dgt)) as max_length
12 FROM
13     dbo.stg_dgt_type
14 UNION ALL
15     -- Anàlisi de la qualitat de les dades de des_type_dgt
16 SELECT
17     'des_type_dgt' AS columna,
18     COUNT(des_type_dgt) AS no_nulos,
19     COUNT(distinct des_type_dgt) AS valores_unicos,
20    min(length(des_type_dgt)) as min_length,
21    max(length(des_type_dgt)) as max_length
22 FROM
23     dbo.stg_dgt_type
24 UNION ALL
25     -- Anàlisi de la qualitat de les dades de cod_subtype_dgt
26 SELECT
27     'cod_subtype_dgt' AS columna,
28     COUNT(cod_subtype_dgt) AS no_nulos,
29     COUNT(distinct cod_subtype_dgt) AS valores_unicos,
30    min(length(cod_subtype_dgt)) as min_length,
```

Data Output

Messages

Notifications

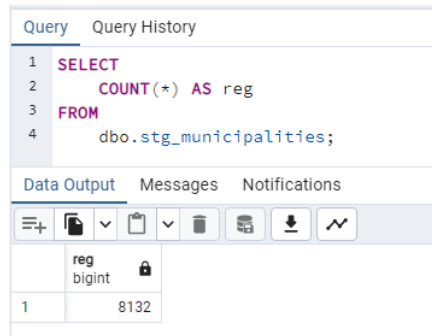
	columna text	no_nulos bigint	nulos bigint	porcentaje_nulos numeric	valores_unicos bigint	min_length integer	max_length integer
1	cod_type_dgt	112	0	0.00	8	1	1
2	des_type_dgt	112	0	0.00	9	5	22
3	cod_subtype_dgt	112	0	0.00	111	2	2
4	des_subtype_dgt	112	0	0.00	110	4	37

Observem que no hi ha cap valor nul per ninguna de les columnes

▪ Taula *stg_municipalities*

De la taula *stg_municipalities* cal fer un anàlisi de la qualitat de les dades que inclogui la volumetria total, l'anàlisi de les columnes de tipus alfanumèric, numèric i data (volumetria i percentatge de valors nuls per columna, valors únics, longituds mínimes i màximes).

En primer lloc, realitzem un recompte del nombre total de registres de la taula:



The screenshot shows the SQL Server Enterprise Manager interface. The 'Query' tab is active, displaying the following SQL query:

```
1 SELECT
2     COUNT(*) AS reg
3 FROM
4     dbo.stg_municipalities;
```

Below the query, the 'Data Output' tab is active, showing the results of the query. The results are displayed in a table with two columns: 'reg' and 'bigint'. The first row shows the value '8132'.

	reg	bigint
1		8132

Obtenim un total de 8132 registres.

Al tenir tantes columnes he decidit dividir l'anàlisi en columnes de tipus *varchar* i les de tipus numèric. Per les de tipus *varchar*, hi ha les columnes: *cod_geo*, *cod_ine*, *cod_prov*, *provincia*, *name_muni*, *surface*, *cod_ine_capital*, *capital*, *longitude_etr89* i *latitude_etr89*. L'script per realitzar l'anàlisi és el següent:

```

WITH total_rows AS(
SELECT
    COUNT(*) AS total
FROM
    dbo.stg_municipalities
),
col_analysis AS(
SELECT
    'cod_geo' AS columna,
    COUNT(cod_geo) AS no_null,
    COUNT(distinct cod_geo) AS unique_values,
    MIN(LENGTH(cod_geo)) AS min_length,
    MAX(LENGTH(cod_geo)) AS max_length
FROM
    dbo.stg_municipalities
UNION ALL
SELECT
    'cod_ine' AS columna,
    COUNT(cod_ine) AS no_null,
    COUNT(distinct cod_ine) AS unique_values,
    MIN(LENGTH(cod_ine)) AS min_length,
    MAX(LENGTH(cod_ine)) AS max_length
FROM
    dbo.stg_municipalities
UNION ALL
SELECT
    'cod_prov' AS columna,
    COUNT(cod_prov) AS no_null,
    COUNT(distinct cod_prov) AS unique_values,
    MIN(LENGTH(cod_prov)) AS min_length,
    MAX(LENGTH(cod_prov)) AS max_length
FROM
    dbo.stg_municipalities
UNION ALL
SELECT
    'provincie' AS columna,
    COUNT(provincie) AS no_null,
    COUNT(distinct provincie) AS unique_values,
    MIN(LENGTH(provincie)) AS min_length,
    MAX(LENGTH(provincie)) AS max_length
FROM
    dbo.stg_municipalities
UNION ALL
SELECT
    'name_muni' AS columna,
    COUNT(name_muni) AS no_null,
    COUNT(distinct name_muni) AS unique_values,
    MIN(LENGTH(name_muni)) AS min_length,
    MAX(LENGTH(name_muni)) AS max_length
FROM
    dbo.stg_municipalities
UNION ALL
SELECT
    'surface' AS columna,
    COUNT(surface) AS no_null,
    COUNT(distinct surface) AS unique_values,
    MIN(LENGTH(surface)) AS min_length,
    MAX(LENGTH(surface)) AS max_length
FROM
    dbo.stg_municipalities
UNION ALL

```

```

SELECT
    'cod_ine_capital' AS columna,
    COUNT(cod_ine_capital) AS no_null,
    COUNT(distinct cod_ine_capital) AS unique_values,
    MIN(LENGTH(cod_ine_capital)) AS min_length,
    MAX(LENGTH(cod_ine_capital)) AS max_length
FROM
    dbo.stg_municipalities

UNION ALL

SELECT
    'capital' AS columna,
    COUNT(capital) AS no_null,
    COUNT(distinct capital) AS unique_values,
    MIN(LENGTH(capital)) AS min_length,
    MAX(LENGTH(capital)) AS max_length
FROM
    dbo.stg_municipalities

UNION ALL

SELECT
    'latitude_etrs89' AS columna,
    COUNT(latitude_etrs89) AS no_null,
    COUNT(distinct latitude_etrs89) AS unique_values,
    MIN(LENGTH(latitude_etrs89)) AS min_length,
    MAX(LENGTH(latitude_etrs89)) AS max_length
FROM
    dbo.stg_municipalities

UNION ALL

SELECT
    'longitude_etrs89' AS columna,
    COUNT(longitude_etrs89) AS no_null,
    COUNT(distinct longitude_etrs89) AS unique_values,
    MIN(LENGTH(longitude_etrs89)) AS min_length,
    MAX(LENGTH(longitude_etrs89)) AS max_length
FROM
    dbo.stg_municipalities
)

SELECT
    ca.columna,
    tr.total AS total_reg,
    ca.no_null,
    ROUND(100.0 * (tr.total - ca.no_null) / tr.total, 2) AS
null_percentage,
    ca.unique_values,
    ca.min_length,
    ca.max_length
FROM col_analysis ca
CROSS JOIN total_rows tr;

```

Aquest, el que fa és crear dues taules “virtuals” (CTE), una anomenada *total_rows* que conté únicament el recompte total de registres de la taula *stg_municipalities*; i la segona, anomenada *col_analysis*, que realitza l’anàlisi sobre les columnes. Aquesta segona, utilitza l’estructura

```

SELECT
    '<columna>' AS columna,
    COUNT(<columna>) AS no_null,
    COUNT(distinct <columna>) AS unique_values,
    MIN(LENGTH(<columna>)) AS min_length,
    MAX(LENGTH(<columna>)) AS max_length
FROM
    dbo.stg_municipalities

```

que crea una fila amb les columnes *columna* que conté el nom de la columna per la que es realitza el càlcul, *no_null* que realitza el recompte de registres no nuls de la columna, *unique_values* que realitza el recompte de valors únics de la columna (amb la clàusula *distinct*), i *min_length* i *max_length* que calculen la longitud mínima i màxima de la columna, respectivament.

S'utilitza aquesta estructura lògica per cada columna que es vol analitzar i s'uneixen amb *UNION ALL*.

Finalment, es selecciona la taula *col_analysis* al *FROM*, amb un *CROSS JOIN* de la taula *total_rows*, per afegir a cada fila el recompte total i realitzar els càlculs de percentatge de nuls.

A la clàusula *SELECT* es seleccionen el nom de la columna, el total de registres, el recompte de no nuls, el percentatge de no nuls, el recompte de valors únics i les longituds mínima i màxima.

Obtenim el següent resultat:

	columna text	total_reg bigint	no_null bigint	null_percentage numeric	unique_values bigint	min_length integer	max_length integer
1	longitude_etr89	8132	8132	0.00	20	1	3
2	latitude_etr89	8132	8132	0.00	12	2	2
3	capital	8132	8132	0.00	8069	2	47
4	cod_ine_capital	8132	8132	0.00	8132	11	11
5	surface	8132	8132	0.00	5711	1	6
6	name_muni	8132	8132	0.00	8115	2	47
7	cod_geo	8132	8132	0.00	8124	5	5
8	cod_ine	8132	8132	0.00	8132	11	11
9	cod_prov	8132	8132	0.00	52	2	2
10	provincia	8132	8132	0.00	52	4	22

En primer lloc veiem que no hi ha cap valor nul en ninguna de les columnes. El percentatge de nuls en totes les columnes és 0%. Per cada columna podem fer el següent anàlisi:

- Longitude_etr89: Hi ha 20 valors únics. Això passa perquè les dades s'han importat com a *number* des de *Pentaho*, perdent els decimals, de manera que no pren molts valors diferents. A més, veiem que la longitud mínima és de 1, que és normal (al no haver-hi decimals), i la màxima de 3, que podria semblar estrany, però es correspon a les longituds negatives, que inclou el signe “-” com un caràcter més.
- Latitude_etr89: Hi ha 12 valors únics, normal al tractar-se de la coordenada de latitud, sense decimals (ja que ha passat el mateix que amb la longitud), que pren pocs valors diferents. Les longituds mínima i màxima són les dues de 2, normal ja que les latituds d'Espanya estan al voltant de 40 (28 a les Illes Canàries).

- Capital i cod ine capital: En aquest cas veiem que hi ha 8132 valors diferents del codi de la capital, com esperaríem, mentre que hi ha 8069 capitals diferents. Hi ha varies capitals que es repeteixen. Entenent la capital del municipi com la seu de l'òrgan de govern del municipi, això implicaria que hi ha diversos municipis que comparteixen el mateix nom per la capital, tot i que sigui diferent (diferent codi ine). Pel que fa a les longituds, també són coherents amb els tipus de valors. La longitud màxima de la capital de 47 caràcter pot semblar estranya, però es correspon a "Sant Vicent del Raspeig/San Vicente del Raspeig".
- Surface: Té un total de 5711 valors únics. Altra vegada, aquest valor s'havia importat com a *number*, i ha perdut els decimals, fent que les superfícies dels municipis més petits es puguin repetir. Per la superfície veiem que la longitud mínima és de 1, i podria semblar un error, però si filtrem pels municipis amb una superfície de 1 xifra de longitud obtenim el següent:

Data Output Messages Notifications		
	name_muni character varying (50)	surface character varying (50)
1	Emperador	3
2	Llocnou de la Corona	1
3	Usansolo	0
4	Maleján	8

I veiem que hi ha 3 pobles amb una superfície molt petita. Però, tot i així, hi ha un amb una superfície de 0, que sí que és un error.

- Name_muni i cod_geo: Per la columna *name_muni* tenim 8115 valors únics. Això es deu a que hi ha municipis amb el mateix nom i unes longituds mínimes i màximes coherents. En canvi, per *cod_geo*, veiem que hi ha 8124 valors únics. No hauria de ser així ja que cada municipi hauria de tenir un codi diferent. Si agrupem els municipis pel codi i filtrem pels que tenen un recompte major de 1, obtenim el següent:

Query Query History		
1	SELECT	
2	cod_geo,	
3	COUNT(*)	
4	FROM dbo.stg_municipalities	
5	GROUP BY cod_geo	
6	HAVING COUNT(*) > 1	
7		

Data Output Messages Notifications		
	cod_geo character varying (5)	count bigint
1	00000	9

Tot i no tenir valors nuls a *cod_geo*, hi ha 9 municipis que tenen un codi de 00000, i es tracta d'un error. Pel que fa a la longitud mínima i màxima, no s'observa cap incoherència ja que tots tenen 5 caràcters.

- Provincia i cod_prov: Finalment, veiem que hi ha 52 valors únics per les províncies i el codi de les províncies, que es corresponen a les 52 províncies d'Espanya (comptant també Ceuta i Melilla, tot i no estar subjectes al règim provincial, però estan incloses com a província a la taula). I longituds mínimes i màximes coherents (per *cod_prov* tots els valors tenen 2 caràcters).

Realitzem la mateixa anàlisi per les columnes de la taula *stg_municipalities* de tipus *integer*.
L'script utilitzat és el següent:

```
WITH total_rows AS(
SELECT
    COUNT(*) AS total_reg
FROM
    dbo.stg_municipalities
),
col_analysis AS(
SELECT
    'population_muni' AS columna,
    COUNT(population_muni) AS no_null,
    COUNT(distinct population_muni) AS unique_values,
    MIN(LENGTH(CAST(population_muni AS varchar))) AS min_length,
    MAX(LENGTH(CAST(population_muni AS varchar))) AS max_length
FROM
    dbo.stg_municipalities
UNION ALL
SELECT
    'perimeter' AS columna,
    COUNT(perimeter) AS no_null,
    COUNT(distinct perimeter) AS unique_values,
    MIN(LENGTH(CAST(perimeter AS varchar))) AS min_length,
    MAX(LENGTH(CAST(perimeter AS varchar))) AS max_length
FROM
    dbo.stg_municipalities
UNION ALL
SELECT
    'population_capital' AS columna,
    COUNT(population_capital) AS no_null,
    COUNT(distinct population_capital) AS unique_values,
    MIN(LENGTH(CAST(population_capital AS varchar))) AS min_length,
    MAX(LENGTH(CAST(population_capital AS varchar))) AS max_length
FROM
    dbo.stg_municipalities
UNION ALL
SELECT
    'altitude' AS columna,
    COUNT(altitude) AS no_null,
    COUNT(distinct altitude) AS unique_values,
    MIN(LENGTH(CAST(altitude AS varchar))) AS min_length,
    MAX(LENGTH(CAST(altitude AS varchar))) AS max_length
FROM
    dbo.stg_municipalities
)

SELECT
    ca.columna,
    tr.total_reg,
    ca.no_null,
    ROUND(100.0 * (tr.total_reg - ca.no_null) / tr.total_reg, 2) AS
null_percentage,
    ca.unique_values,
    ca.min_length,
    ca.max_length
FROM col_analysis ca
CROSS JOIN total_rows tr;
```


- Perimeter: Hi ha 7590 valors únics, un resultat coherent ja que pels municipis més petits és molt probable que alguns valors de perímetre es repeteixin. La longitud màxima també es coherent, però veiem que hi ha una longitud mínima de 1, que no pot ser correcte. Si filtrem pels municipis amb una perímetre amb una longitud de 1 caràcter obtenim el següent:

Query		Query History	
1	SELECT		
2	name_muni,		
3	cod_geo,		
4	surface,		
5	perimeter		
6	FROM		
7	dbo.stg_municipalities		
8	WHERE		
9	LENGTH(CAST(perimeter AS varchar)) = 1		
10			
Data Output			
Messages			
Notifications			
	name_muni character varying (50)	cod_geo character varying (5)	surface character varying (50)
	perimeter integer		
1	Usansolo	00000	0

Es tracta de Usansolo, el municipi amb una superfície de 0, que també és un dels que té el codi 00000.

▪ Taula *stg_vehicles*

Per la taula *stg_vehicles* realitzem la mateixa anàlisi, determinant també la completesa de les columnes de tipus booleà (*new_used*). He dividit l'anàlisi entre les columnes de tipus *varchar*, les de tipus *integer* i data, i la de tipus *boolean*.

Començant per les columnes de tipus *varchar*, l'script que he utilitzat és el següent:

```

WITH total_rows AS(
SELECT
    COUNT(*) AS total_reg
FROM
    dbo.stg_vehicles
),
col_analysis AS(
SELECT
    'municipality' AS columna,
    COUNT(municipality) AS no_null,
    COUNT(distinct municipality) AS unique_values,
    MIN(LENGTH(municipality)) AS min_length,
    MAX(LENGTH(municipality)) AS max_length
FROM
    dbo.stg_vehicles
UNION ALL
SELECT
    'brand' AS columna,
    COUNT(brand) AS no_null,
    COUNT(distinct brand) AS unique_values,
    MIN(LENGTH(brand)) AS min_length,
    MAX(LENGTH(brand)) AS max_length
FROM
    dbo.stg_vehicles
UNION ALL
SELECT
    'model' AS columna,
    COUNT(model) AS no_null,
    COUNT(distinct model) AS unique_values,
    MIN(LENGTH(model)) AS min_length,
    MAX(LENGTH(model)) AS max_length
FROM
    dbo.stg_vehicles
UNION ALL
SELECT
    'subtype' AS columna,
    COUNT(subtype) AS no_null,
    COUNT(distinct subtype) AS unique_values,
    MIN(LENGTH(subtype)) AS min_length,
    MAX(LENGTH(subtype)) AS max_length
FROM
    dbo.stg_vehicles
)

SELECT
    ca.columna,
    tr.total_reg,
    ca.no_null,
    ROUND(100.0 * (tr.total_reg - ca.no_null) / tr.total_reg, 2) AS
null_percentage,
    ca.unique_values,
    ca.min_length,
    ca.max_length
FROM col_analysis ca
CROSS JOIN total_rows tr;

```

Igual que per la taula anterior. En aquest cas, obtenim els següents resultats:

Data Output Messages Notifications								
	columna text	total_reg bigint	no_null bigint	null_percentage numeric	unique_values bigint	min_length integer	max_length integer	
1	subtype	1721704	1721704	0.00	47	2	2	
2	brand	1721704	1721704	0.00	127	2	22	
3	model	1721704	1721704	0.00	1045	1	22	
4	municipality	1721704	1721704	0.00	27	5	5	

La columna *total_reg* conté la volumetria total de la taula, obtingut de la taula virtual *total_rows*. Observem que no hi ha valors nuls per cap de les columnes. Per cada una de les columnes tenim el següent:

- Subtype: Hi ha un total de 47 valors únics, un resultat coherent, i unes longituds mínima i màxima de 2, ja que el codi dels subtipus estan formats per dos caràcters.
- Municipality: Hi ha 27 valors únics. Aquest fet pot semblar estrany ja que són molts pocs municipis diferents, però no es pot considerar com un error, ja que són les dades de les que disposem, excepte si es tracta de dades reals, que s'hauria de comprovar que les dades siguin coherents (que no hi hagi més vehicles que habitants, per exemple). La longitud mínima i màxima és de 5, que és correcte.
- Brand: Hi ha un total de 127 valors únics, i és molt probable que en la taula hi hagi 127 marques diferents. La longitud mínima de *brand* es correspon a "MG" i la màxima a "CARROCERA CASTROSUA SA", per tant també és correcte.
- Model: Hi ha un total de 1045 valors únics, que sembla correcte. La longitud mínima de 1 pot semblar estranya però es correspon a "T" (marca Renault).

Per les columnes de tipus *integer* i data utilitzem el següent script:

```

WITH total_rows AS(
SELECT
    COUNT(*) AS total_reg
FROM
    dbo.stg_vehicles
),
col_analysis AS(
SELECT
    'id' AS columna,
    COUNT(id) AS no_null,
    COUNT(distinct id) AS unique_values,
    MIN(LENGTH(CAST(id AS varchar))) AS min_length,
    MAX(LENGTH(CAST(id AS varchar))) AS max_length
FROM
    dbo.stg_vehicles
UNION ALL
SELECT
    'registration_date' AS columna,
    COUNT(registration_date) AS no_null,
    COUNT(distinct registration_date) AS unique_values,
    MIN(LENGTH(CAST(registration_date AS varchar))) AS min_length,
    MAX(LENGTH(CAST(registration_date AS varchar))) AS max_length
FROM
    dbo.stg_vehicles
)

SELECT
    ca.columna,
    tr.total_reg,
    ca.no_null,
    ROUND(100.0 * (tr.total_reg - ca.no_null) / tr.total_reg, 2) AS
null_percentage,
    ca.unique_values,
    ca.min_length,
    ca.max_length
FROM col_analysis ca
CROSS JOIN total_rows tr;

```

Realitzant el *CAST* de les dades a *varchar*. En aquest cas, obtenim el següent resultat:

	columna text	total_reg bigint	no_null bigint	null_percentage numeric	unique_values bigint	min_length integer	max_length integer
1	registration_date	1721704	1721704	0.00	694	10	10
2	id	1721704	1721704	0.00	1721704	1	7

Altra vegada, no hi ha cap valor nul per cap columna. A més, per la columna *registration_date* tenim 694 valors únics, ja que comença a la data '2020-01-01'. Tots els valors de la columna tenen 10 caràcters, coherent amb el format 'YYYY-MM-DD'. Per la *id* tenim que tots els valors són únics, correcte al tractar-se d'un identificador del vehicle, ja que no hi pot haver repetits. La longitud mínima és 1 (que es correspon amb 1) i la màxima és 7 (corresponent a 1721704, l'últim vehicle registrat).

Finalment, per la columna *new_used*, de tipus booleà utilitzem el següent script:

```

WITH total_rows AS(
SELECT
    COUNT(*) AS total_reg
FROM
    dbo.stg_vehicles
),
col_analysis AS(
SELECT
    'new_used' AS columna,
    COUNT(new_used) AS no_null,
    COUNT(distinct new_used) AS unique_values
FROM
    dbo.stg_vehicles
)

SELECT
    ca.columna,
    tr.total_reg,
    ca.no_null,
    ROUND(100.0 * (tr.total_reg - ca.no_null) / tr.total_reg, 2) AS
null_percentage,
    ca.unique_values
FROM col_analysis ca
CROSS JOIN total_rows tr;

```

En aquest cas, al tractar-se d'un booleà no cal calcula les longituds màxima i mínima, i només calculem el nombre i percentatge de nuls i el nombre de valors únics.

Obtenim el següent resultat:

Data Output Messages Notifications						
	columna	total_reg	no_null	null_percentage	unique_values	
	text	bigint	bigint	numeric	bigint	
1	new_used	1721704	1721704	0.00	2	

Observem que no hi ha valors nuls i que hi ha un total de 2 valors únics (true i false).

Una vegada analitzades les dues taules, hem vist que hi ha un error greu d'unicitat a la columna *cod_geo* de la taula *stg_municipalities*, ja que es repeteix varies vegades el codi 00000. A més hi ha un municipi amb perímetre i superfície de 0, un fet físicament impossible. Per dur a terme un correcte anàlisi, es podria valorar l'eliminació d'aquests municipis, ja que poden influir en diversos càlculs.

A més, com hem vist, algunes dades, com la longitud i la latitud dels municipis i la superfície han perdut els decimals al importar-se al *CSV file input* com a *number*, i no com *string*. Per solucionar-ho, ens hem d'assegurar que les dades s'importen en el format i tipus correcte, començant per modificar la transformació *IN_MUNICIPALITIES* de la següent forma:

		File encoding UTF-8					
#	Name	Type	Format	Length	Precision	Currency	D
7	surface	String	#,.	15	0	\$	.
8	perimeter	Integer	#	15	0	\$	.
9	cod_in_e_capital	String	#	15	0	\$	.
10	capital	String		42		\$	.
11	population_capital	Integer	#	15	0	\$	.
12	longitude_etr89	String	#,.	15	0	\$	.
13	latitude_etr89	String	#,.	15	0	\$	.
14	altitude	Integer	#	15	0	\$	.

Executem i observem el canvi a la taula de la base de dades.

Data Output Messages Notifications											
		population_muni integer	surface character varying (50)	perimeter integer	cod_in_e_capital character varying	capital character varying (50)	population_capital integer	longitude_etr89 character varying (50)	latitude_etr89 character varying (50)	altitude integer	
1	)	2975	1994,5872	35069	01001000101	Alegria-Dulantzi	2860	-2,51243731	42,83981158	568	
2		10313	9629,68	65381	01002000201	Amurrio	9238	-3,00007326	43,05427776	219	
3		1409	7308,96	42097	01003000601	Ibarra	758	-2,56540037	43,05119653	333	
4		1832	2728,73	22886	01004000101	Artziniega	1697	-3,12791718	43,12084358	210	
5		232	1297,27	24707	01006000101	Armiñón	113	-2,87183475	42,72326199	467	
6		1043	5753,18	67180	01008000501	Durana	389	-2,63878816	42,89116171	528	
7		1609	6510,76	53754	01009000401	Araia	1207	-2,31670754	42,88968768	602	
8		2918	14096,94	64211	01010002001	Arespalditza/Respaldiza	414	-3,04306746	43,07768283	306	
9	ieta	295	950,39	18666	01011000101	Baños de Ebro/Mañueta	295	-2,67863583	42,52925602	421	
10		897	9729,68	57551	01013001401	Ozaeta	210	-2,49407452	42,91537538	575	
11		451	3574,75	38696	01014000101	Berantevilla	301	-2,85971414	42,68227191	471	
12		540	13048,2	85964	01016000301	Bernedo	176	-2,49825439	42,62676989	710	
13		1072	8536,0875	53461	01017000501	Santa Cruz de Campezo/Santikurutze Kanpezu	787	-2,35207693	42,66971209	578	
14		1821	10209,12	48687	01018000901	Gopegi	272	-2,73033002	42,96210609	585	
15		174	1251,599	15980	01019000101	Kripan	174	-2,51621636	42,59082439	691	

Veiem com ara aquestes columnes sí que conserven els decimals.

Exercici 3. Càrrega d'una taula de fets i les seves dimensions a la base de dades

Comencem creant la transformació de la dimensió *DIM_DGT_TYPE*. Per fer-ho, es llegeixen les dades de la taula *stg_dgt_type* amb un *Table input*, s'afageix una seqüència (*primary key*) amb el component *Add sequence* i, finalment, s'insereixen les dades a una nova taula *dim_dgt_type* de la bse de dades de *PostgreSQL* amb un *Table output*.

Al *table input* s'escriu una sentència SQL per seleccionar les dades que volem carregar, en aquest cas seleccionem totes les columnes i passem a majúscules la columna *cod_subtype_dg* per evitar incoherències.

Table input

Step name: Table input (stg_dgt_types)

Connection: cn_stg Edit... New... Wizard...

SQL

```
SELECT DISTINCT
  cod_type_dgt,
  des_type_dgt,
  UPPER(cod_subtype_dgt) as cod_subtype_dgt,
  des_subtype_dgt
FROM
  dbo.stg_dgt_type
```

Line 1 Column 0

Store column info in step meta data ☐

Enable lazy conversion ☐

Replace variables in script? ☐

Insert data from step

Execute for each row? ☐

Limit size 0

Help OK Preview Cancel

Amb *Add sequence*, creem una seqüència anomenada *pk*, de la següent manera.

Add sequence

Step name: Add sequence

Name of value: pk

Use a database to generate the sequence

Use DB to get sequence? ☐

Connection: cn_stg Edit... New... Wizard...

Schema name: Schemas...

Sequence name: SEQ_ Sequences...

Use a transformation counter to generate the sequence

Use counter to calculate sequence? ☒

Counter name (optional):

Start at value: 1

Increment by: 1

Maximum value: 999999999

Help OK Cancel

Creem la taula *dim_dgt_type* a la base de dades amb l'script proporcionat a l'enunciat.

Finalment, amb el *table output*, ens connectem a la base de dades, seleccionem la taula i els camps i assignem la seqüència *pk* al camp *id_dgt_type*.

Table output

Step name: Table output (dim_dgt_type)

Connection: cn_stg Edit... New... Wizard...

Target schema: dbo Browse...

Target table: dim_dgt_type Browse...

Commit size: 1000

Truncate table ☐

Ignore insert errors ☐

Specify database fields ☒

Main options Database fields

Fields to insert:

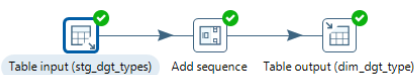
#	Table field	Stream field
1	cod_type_d...	cod_type_dgt
2	des_type_dgt	des_type_dgt
3	cod_subtyp...	cod_subtype...
4	des_subtyp...	des_subtype...
5	id_dgt_type	pk

Get fields

Enter field mapping

Help OK Cancel SQL

Executem la transformació i comprovem la correcta càrrega de les dades a la taula:



Execution Results

Logging | Execution History | Step Metrics | Performance Graph | Metrics | Preview data

2025/12/12 17:19:06 - RepositoriesMeta - Reading repositories XML file: F:\Kettle\kettle\repositories.xml
 2025/12/12 17:19:47 - RepositoriesMeta - Reading repositories XML file: F:\Kettle\kettle\repositories.xml
 2025/12/12 17:19:55 - RepositoriesMeta - Reading repositories XML file: F:\Kettle\kettle\repositories.xml
 2025/12/12 17:19:55 - Spoon - Connected to metastore : PR3, added to delegating metastore
 2025/12/12 17:19:55 - RepositoriesMeta - Reading repositories XML file: F:\Kettle\kettle\repositories.xml
 2025/12/12 17:20:27 - Spoon - Running transformation using the Kettle execution engine
 2025/12/12 17:20:27 - Spoon - Transformation opened.
 2025/12/12 17:20:27 - Spoon - Launching transformation [DIM_DGT_TYPES]...
 2025/12/12 17:20:27 - Spoon - Started the transformation execution.
 2025/12/12 17:20:28 - DIM_DGT_TYPES - Dispatching started for transformation [DIM_DGT_TYPES]
 2025/12/12 17:20:28 - Table output (dim_dgt_type).0 - Connected to database [cn_stg] (commit=1000)
 2025/12/12 17:20:28 - Table input (stg_dgt_types).0 - !TableInput.Log.FinishedReadingQuery!
 2025/12/12 17:20:28 - Table input (stg_dgt_types).0 - Finished processing (I=111, O=0, R=0, W=111, U=0, E=0)
 2025/12/12 17:20:28 - Add sequence.0 - Finished processing (I=0, O=0, R=111, W=111, U=0, E=0)
 2025/12/12 17:20:28 - Table output (dim_dgt_type).0 - Finished processing (I=0, O=111, R=111, W=111, U=0, E=0)
 2025/12/12 17:20:28 - Spoon - The transformation has finished!!

Data Output | Messages | Notifications

	id_dgt_type [PK] integer	cod_type_dgt character varying (1)	des_type_dgt character varying (50)	cod_subtype_dgt character varying (2)	des_subtype_dgt character varying (100)
1	1	9	R Y S	SC	SEMIREMOLQUE HORMIGONERA
2	2	9	R Y S	R3	REMOLQUE FURGÓN
3	3	9	CICLOMOTORES	90	CICLOMOTOR DE 2 RUEDAS
4	4	9	R Y S	S9	SEMIREMOLQUE CANTERA
5	5	5	MOTOCICLETAS	51	MOTOCICLETA CON SIDECAR
6	6	9	R Y S	RE	REMOLQUE DE GRÚA
7	7	8	TRACTORES INDUSTRIALES	81	TRACTOCAMIÓN
8	8	1	CAMIONES	06	CAMIÓN JAULA
9	9	9	R Y S	SB	SEMIREMOLQUE VIAJEROS O AUTOBÚS
10	10	0	OTROS vehiculos	7J	PALA CARGADORA-RETROEXCAVADORA

Per la dimensió *DIM_DATE*, utilitzem el component de *Spoon*, *Execute SQL script*, amb l'script proporcionat a l'enunciat, que genera les dates dels de '2020-01-01' fins la data actual, i insereix a la taula *dim_date* la informació de la data en les columnes *date*, *year*, *month*, *day*.

Execute SQL script

Step name: Execute SQL script (dim_date)

Connection: cn_stg

SQL script to execute. (statements separated by ;) Question marks will be replaced by arguments.

```
DELETE FROM dbo.dim_date;
INSERT INTO dbo.dim_date(date, year, month, day)
SELECT
d,
EXTRACT(YEAR FROM d)::INT,
EXTRACT(MONTH FROM d)::INT,
EXTRACT(DAY FROM d)::INT
FROM generate_series('2020-01-01'::DATE, CURRENT_DATE, '1 day') AS d;
```

Line 8 Column 69

Execute for each row? ☐
 Execute as a single statement? ☐
 Variable substitution? ☐
 Bind parameters? ☐
 Quote Strings? ☐

Parameters:

#	Field name to be used as argument
1	

Field to contain insert stats:
 Field to contain Update stats:
 Field to contain Delete stats:
 Field to contain Read stats:

Help OK Cancel Get fields

Executem i comprovem que les dades s'han carregat correctament:

The screenshot shows the Kettle execution results on the left and the query output on the right.

Execution Results

- 2025/12/13 16:24:31 - Spoon - Running transformation using the Kettle execution engine
- 2025/12/13 16:24:31 - Spoon - Transformation opened.
- 2025/12/13 16:24:31 - Spoon - Launching transformation [DIM_DATE]...
- 2025/12/13 16:24:31 - Spoon - Started the transformation execution.
- 2025/12/13 16:24:32 - DIM_DATE - Dispatching started for transformation [DIM_DATE]
- 2025/12/13 16:24:32 - Execute SQL script (dim_date).0 - Finished reading query, closing connection.
- 2025/12/13 16:24:32 - Execute SQL script (dim_date).0 - Finished processing (I=0, O=0, R=0, W=1, U=0, E=0)
- 2025/12/13 16:24:32 - Spoon - The transformation has finished!!

Query

```
1 SELECT *
2 FROM
3     dbo.dim_date
4 ORDER BY date ASC;
```

Data Output

	date [PK] date	year integer	month integer	day integer
1	2020-01-01	2020	1	1
2	2020-01-02	2020	1	2
3	2020-01-03	2020	1	3
4	2020-01-04	2020	1	4
5	2020-01-05	2020	1	5
6	2020-01-06	2020	1	6
7	2020-01-07	2020	1	7
8	2020-01-08	2020	1	8

Per la dimensió *DIM_VEHICLE* seguim el mateix procediment que per la dimensió de *dgt_type*, amb un *Table input*, un *Add sequence* i un *Table output*.

El *table input* recull les dades de la taula *stg_vehicles* per obtenir els diferents vehicles (marca, model i subtipus) que hi ha registrats.

The screenshot shows the 'Table input' dialog box in Kettle. The 'Step name' is 'Table input (stg_vehicle)' and the 'Connection' is 'cn_stg'. The SQL query is:

```
SELECT DISTINCT
    brand,
    model,
    subtype
FROM
    dbo.stg_vehicles;
```

Line 6 Column 18

Store column info in step meta data ☐

Enable lazy conversion ☐

Replace variables in script? ☐

Insert data from step

Execute for each row? ☐

Limit size

Buttons: ? Help, OK, Preview, Cancel

Mitjançant el *add sequence* creem una seqüència per utilitzar com a id dels vehicles.

The screenshot shows the 'Add sequence' dialog box in Kettle. The 'Step name' is 'Add sequence' and the 'Name of value' is 'id'. The 'Connection' is 'cn_stg'. The 'Schema name' is 'SEQ_' and the 'Sequence name' is 'SEQ_'. The 'Use a transformation counter to generate the sequence' checkbox is checked. The 'Counter name (optional)' is empty. The 'Start at value' is '1', the 'Increment by' is '1', and the 'Maximum value' is '999999999'.

Buttons: ? Help, OK, Cancel

Es crea la taula *dim_vehicle* a partir de l'script proporcionat a l'enunciat. I, finalment, amb el *table output*, ens connectem a la base de dades, seleccionem la taula i els camps, assignem la *id* a *vehicle_id*. Executem i comprovem que les dades s'hagin inserit correctament.

The 'Table output' dialog box shows the configuration for the 'Table output (dim_vehicle)' step. The connection is 'cn_stg', the target schema is 'dbo', and the target table is 'dim_vehicle'. The commit size is set to 1000. The 'Specify database fields' checkbox is checked. The 'Fields to insert' table is as follows:

#	Table field	Stream field
1	brand	brand
2	model	model
3	subtype	subtype
4	vehicle_id	id

The 'Execution Results' window shows the following log messages:

```

2025/12/13 16:43:46 - Spoon - Running transformation using the Kettle execution engine
2025/12/13 16:43:46 - Spoon - Transformation opened.
2025/12/13 16:43:46 - Spoon - Launching transformation [DIM_VEHICLE]...
2025/12/13 16:43:46 - Spoon - Started the transformation execution.
2025/12/13 16:43:46 - DIM_VEHICLE - Dispatching started for transformation [DIM_VEHICLE]
2025/12/13 16:43:46 - Table output (dim_vehicle).0 - Connected to database [cn_stg] (commit=1000)
2025/12/13 16:43:48 - Table input (stg_vehicle).0 - TableInputLog.FinishedReadingQuery()
2025/12/13 16:43:48 - Table input (stg_vehicle).0 - Finished processing (I=1257, O=0, R=0, W=1257, U=0, E=0)
2025/12/13 16:43:48 - Add sequence.0 - Finished processing (I=0, O=0, R=1257, W=1257, U=0, E=0)
2025/12/13 16:43:48 - Table output (dim_vehicle).0 - Finished processing (I=0, O=1257, R=1257, W=1257, U=0, E=0)
2025/12/13 16:43:48 - Spoon - The transformation has finished!
  
```

Data Output				
vehicle_id	brand	model	subtype	
1	BMW	218D	40	
2	AIXAM	S9	92	
3	YAMAHA	MTN690-U (MT-07)	50	
4	CITROEN	JUMPER BLUEHDI 140 S&S	7A	
5	VOLKSWAGEN	ID.4 PRO 150 KW	40	
6	NISSAN	NV250	20	
7	PEUGEOT	BOXER FURGÓN 335 L2 H2	20	
8	ALFA ROMEO	GIULIA	40	
9	PIAGGIO	MP3 300 HPE	53	
10	CITROEN	C4 BLUEHDI 100 FEEL ED	40	
11	PIAGGIO	LIBERTY	90	
12	SMART	EQ FORTWO COUPE	40	
13	BERGADANA	SPRINTER 910 AMBULAN...	22	
Total rows: 1000 of 1257 Query complete 00:00:00.151				

Per la creació de la taula de fets *fact_vehicle_registration*, es crea la transformació *FACT_VEHICLE_TRANSFORMATION*, que, de manera similar a la transformació de la dimensió anterior, es fa mitjançant un *table input*, un *add sequence* i un *table output*.

En aquest cas, al *table input* s'utilitza un script que pren les dades de la taula *stg_vehicles*, amb un *JOIN* amb *dim_vehicle*, i un altre amb *dim_dgt_type*. A partir d'aquestes taules, obté, el *vehicle_id*, el *id_dgt_type*, *registration_date*, *new_used* i la quantitat (*quantity*) = 1

The 'Table input' dialog box shows the configuration for the 'Table input (stg_vehicles)' step. The connection is 'cn_stg'. The SQL query is as follows:

```

SELECT
  v.vehicle_id,
  dgt.id_dgt_type,
  s.registration_date,
  s.new_used,
  1 quantity
FROM dbo.stg_vehicles s
JOIN dbo.dim_vehicle v ON
  (s.brand = v.brand AND s.model = v.model AND s.subtype = v.subtype)
JOIN dbo.dim_dgt_type dgt ON
  s.subtype = dgt.cod_subtype_dgt;
  
```

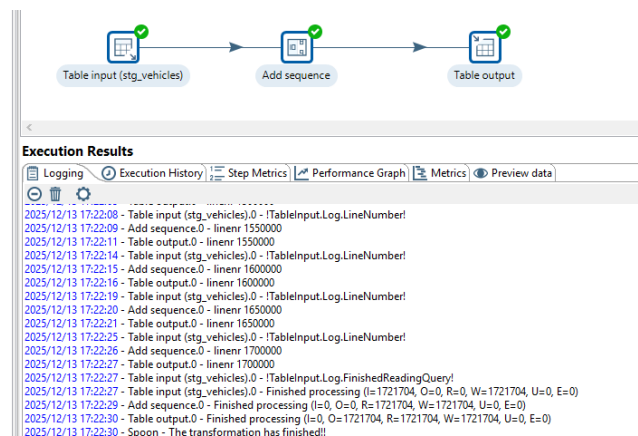
The 'Limit size' is set to 0.

En segon lloc es genera la seqüència *pk* per actual com a id.

Creem la taula *fact_vehicle_transformation* i mitjançant el *table output*, ens connectem a la base de dades, seleccionem la taula i els camps i assignem la seqüència *pk* a *id_registration*.

#	Table field	Stream field
1	vehicle_id	vehicle_id
2	id_dgt_type	id_dgt_type
3	registration_date	registration_date
4	new_used	new_used
5	quantity	quantity
6	id_registration	pk

Executem la transformació i comprovem la correcta inserció de les dades.



Data Output Messages Notifications							
	id_registration [PK] integer	registration_date date	vehicle_id integer	id_dgt_type integer	new_used boolean	quantity integer	
1	1	2020-03-01	769	64	true	1	
2	2	2020-03-01	769	64	true	1	
3	3	2020-03-01	769	64	true	1	
4	4	2020-03-01	769	64	true	1	
5	5	2020-03-01	769	64	true	1	
6	6	2020-03-01	769	64	true	1	
7	7	2020-03-01	769	64	true	1	
8	8	2020-03-01	769	64	true	1	
9	9	2020-03-01	769	64	true	1	
10	10	2020-03-01	769	64	true	1	
11	11	2020-03-01	769	64	true	1	
12	12	2020-03-01	769	64	true	1	
13	13	2020-03-01	769	64	true	1	
Total rows: 1000 of 1721704 Query complete 00:00:02.673							

3.5 Job *JOB_LOAD*

Per crea el job *JOB_LOAD*, introduïm els passos *Start - Set variables - SQL*, seguit de les 4 transformacions de les dimensions i la taula de fets, acabant amb un *Success*. Cada un dels passos el configurem de la següent manera:

Set variables

Job entry name: Set variables

Properties file

Name of properties file:

Variable scope: Valid in the Java Virtual Machine

Settings

Variable substitution? ☒

Variables:

#	Variable name	Value	Variable scope type
1	DIR_IN	F:\Fonts	Valid in the Java Virtual Machine
2	BASE_PATH	F:\PRJ	Valid in the Java Virtual Machine

OK Cancel

SQL

Job entry name: SQL

Connection: cn_stg

SQL from file ☐

SQL filename:

Send SQL as single ☐

Use variable substitution? ☐

SQL Script:

```
TRUNCATE TABLE dbo.fact_vehicle_registration;
DELETE FROM dbo.DIM_DGT_TYPE;
DELETE FROM dbo.DIM_DATE;
DELETE FROM dbo.DIM_VEHCILE;
```

Line 4 Column 27

OK Cancel

Transformation

Entry Name: DIM_VEHCILE

Transformation: /DIM_VEHCILE

Options Logging Arguments Parameters

Run configuration: Pentaho local

Execution

☐ Execute every input row

☐ Clear results rows before execution

☐ Clear results files before execution

☒ Wait for remote transformation to complete

OK Cancel

Transformation

Entry Name: DIM_DATE

Transformation: /DIM_DATE

Options Logging Arguments Parameters

Run configuration: Pentaho local

Execution

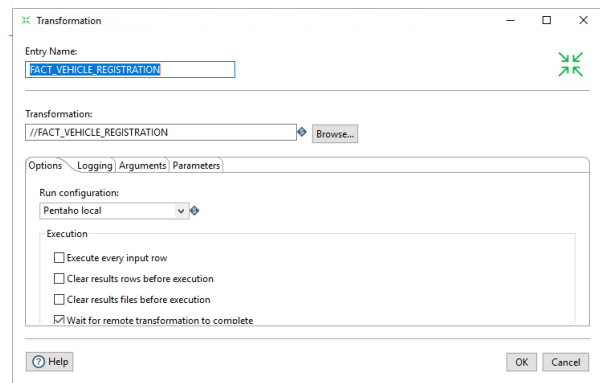
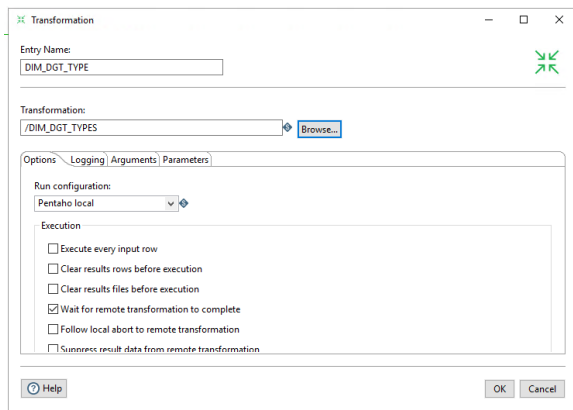
☐ Execute every input row

☐ Clear results rows before execution

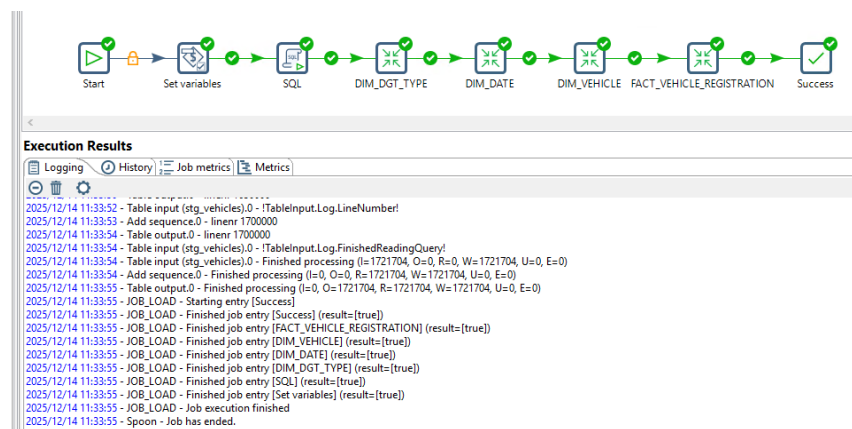
☐ Clear results files before execution

☒ Wait for remote transformation to complete

OK Cancel



Finalment, executem el job per comprovar el funcionament correcte.



3.6 Exercicis a realitzar

Creació de la transformació *DIM_MUNICIPALITY*

La transformació *DIM_MUNICIPALITY* ens permet incloure al model estrella la dimensió de les dades geogràfiques dels vehicles registrats de la taula *stg_municipalities*.

Seguint els passos de les transformacions de les dimensions anteriors, utilitzem un *Table input* per obtenir les dades de la taula *stg_municipalities*, amb una consulta que recull totes les columnes de la taula; un *Add sequence* per crea una seqüència per assignar al *id* del municipi; i, finalment, un *Table output* per carregar les dades a la taula *dim_municipality* de la base de dades de *PostgreSQL*.

L'script per la creació de la taula *dim_municipality* és el següent:

```

DROP TABLE IF EXISTS dbo.dim_municipality;
CREATE TABLE dbo.dim_municipality (
    id_municipality integer PRIMARY KEY,
    cod_ine varchar(15) NOT NULL,
    cod_geo varchar(5),
    cod_prov varchar(2),
    provincie varchar(50),
    name_muni varchar(50),
    population_muni integer ,
    surface varchar(50),
    perimeter integer,
    cod_ine_capital varchar,
    capital varchar(50),
    population_capital integer,
    longitude_etrs89 varchar(50),
    latitude_etrs89 varchar(50),
    altitude integer
);

```

Com hem vist anteriorment, hi ha municipis sense un codi *cod_geo* vàlid, o sense una superfície o perímetre vàlids. He decidit no incloure aquests municipis al obtenir les dades al *table input*, ja que no tenen la informació completa. L'script utilitzat és el següent:

```

SELECT DISTINCT
    cod_ine,
    cod_geo,
    cod_prov,
    provincie,
    name_muni,
    population_muni,
    surface,
    perimeter,
    cod_ine_capital,
    capital,
    population_capital,
    latitude_etrs89,
    longitude_etrs89,
    altitude
FROM
    dbo.stg_municipalities
WHERE
    surface <> '0'
    AND perimeter > 0
    AND cod_geo <> '0'

```

Aquest script filtra les files per les que tenen una superfície diferent de 0, un perímetre major que 0 i un *cod_geo* diferent de '00000'.

El *Table input* queda de la següent manera:

Table input

Step name:

Connection:

SQL

```
SELECT DISTINCT
  cod_line,
  cod_geo,
  cod_prov,
  province,
  name_muni,
  population_muni,
  surface,
  perimeter,
  cod_line_capital,
  capital,
  population_capital,
  latitude_etr89,
  longitude_etr89,
  altitude
FROM
  dbo.stg_municipalities
WHERE
  surface <> '0'
  AND perimeter > 0
  AND cod_geo <> '00000'
```

Line 1 Column 0

Store column info in step meta data ☐

Enable lazy conversion ☐

Replace variables in script? ☐

Insert data from step

Execute for each row? ☐

Limit size

De manera similar als casos anteriors, el component *Add sequence* queda de la següent forma, creant una seqüència anomenada *id*.

Add sequence

Step name:

Name of value:

Use a database to generate the sequence

Use DB to get sequence? ☐

Connection:

Schema name:

Sequence name:

Use a transformation counter to generate the sequence

Use counter to calculate sequence? ☒

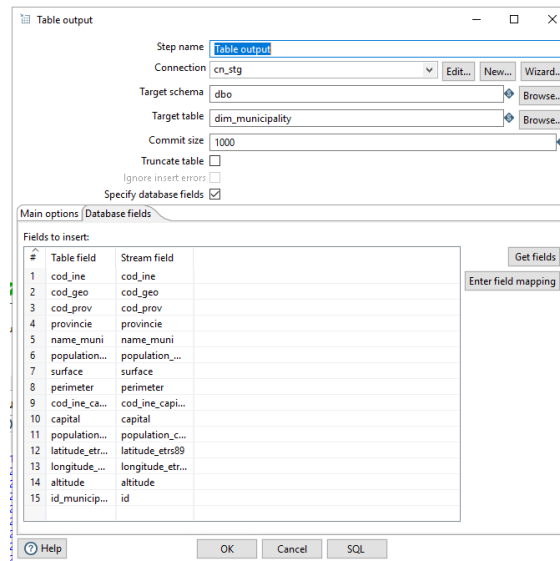
Counter name (optional):

Start at value:

Increment by:

Maximum value:

Finalment, mitjançant el *Table output* ens connectem a la base de dades, seleccionem la taula i assignem la seqüència *id* al camp *id_municipality* de la taula:



Executem la transformació i comprovem la correcta inserció dels elements a la taula.


Table input


Add sequence


Table output

Execution Results

Logging Execution History Step Metrics Performance Graph Metrics Preview data

2025/12/16 13:17:31 - Spoon - Running transformation using the Kettle execution engine

2025/12/16 13:17:32 - Spoon - Transformation opened.

2025/12/16 13:17:32 - Spoon - Launching transformation [DIM_MUNICIPALITY]...

2025/12/16 13:17:32 - Spoon - Started the transformation execution.

2025/12/16 13:17:32 - DIM_MUNICIPALITY - Dispatching started for transformation [DIM_MUNICIPALITY]

2025/12/16 13:17:32 - Table output.0 - Connected to database [cn_stg] (commit=1000)

2025/12/16 13:17:32 - Table input.0 - TableInput.Log.FinishedReadingQuery!

2025/12/16 13:17:32 - Table input.0 - Finished processing (I=0, O=0, R=8123, W=8123, U=0, E=0)

2025/12/16 13:17:32 - Add sequence.0 - Finished processing (I=0, O=0, R=8123, W=8123, U=0, E=0)

2025/12/16 13:17:32 - Table output.0 - Finished processing (I=0, O=8123, R=8123, W=8123, U=0, E=0)

2025/12/16 13:17:32 - Spoon - The transformation has finished!!

	id_municipality [PK] integer	cod_line character varying (15)	cod_geo character varying (5)	cod_prov character varying (2)	provincia character varying (50)	name_muni character varying (50)	population_muni integer	surface charact
1	1	09907000000	09023	09	Burgos	Alfoz de Quintanadueñas	2180	4150,81
2	2	45146000000	45580	45	Toledo	Retamoso de la Jara	109	4824,49
3	3	47039000000	47160	47	Valladolid	Castrillo-Tejeriego	157	3592,23
4	4	47023000000	47096	47	Valladolid	Boecillo	4297	2411,86
5	5	07057000000	07560	07	Illes Balears	Santanyí	12480	12486,3
6	6	48097000000	48950	48	Bizkaia	Zaratamo	1610	1018,3
7	7	28026000000	28130	28	Madrid	Brunete	11014	4894,35
8	8	19252000000	19657	19	Guadalajara	Sayatón	63	4537,53
9	9	09446000000	09814	09	Burgos	Villamedianilla	9	675,825
10	10	14008000000	14080	14	Córdoba	Bealáczár	3138	35600,2
11	11	43106000000	43525	43	Tarragona	El Pinell de Brai	1006	5728,87
12	12	49203000000	49567	49	Zamora	Santa Maria de la Vega	293	1788,17
13	13	31155000000	31465	31	Navarra	Liédena	323	1927,98
14	14	29074000000	29730	29	Màlaga	Montequaque	970	4545,53
15	15	40206000000	40736	40	Segovia	Torre Val de San Pedro	190	4440,54

Total rows: 1000 of 8123 Query complete 00:00:00.378

En aquest cas veiem que hi ha un total de 8123 registres, ja que s'han eliminat els 9 amb codi = '00000' (un d'ells és també el que tenia superfície i perímetre 0).

Adaptació de la transformació *FACT_VEHICLE_REGISTRATION*

Al crear la nova dimensió *DIM_MUNICIPALITY*, necessitem incloure-la a la transformació de la taula de fets. Per fer-ho, modifiquem primer l'script per la creació de la taula a la base de dades.

```

DROP TABLE IF EXISTS dbo.fact_vehicle_registration;
CREATE TABLE dbo.fact_vehicle_registration (
    id_registration INT NOT NULL PRIMARY KEY, -- Clau
    registration_date date NOT NULL , -- Clau forana cap a dim_date (clau natural)
    vehicle_id INT NOT NULL, -- Clau forana cap a dim_vehicle
    id_dgt_type INT NOT NULL, -- Clau forana cap a dim_dgt_type
    id_municipality INT NOT NULL, -- Clau forana cap a dim_municipality
    new_used BOOLEAN, -- Si és nou o usat
    quantity INT NOT NULL DEFAULT 1, -- Mesura (1 per cada registre d'alta)
    FOREIGN KEY (registration_date) REFERENCES dbo.dim_date(date),
    FOREIGN KEY (vehicle_id) REFERENCES dbo.dim_vehicle(vehicle_id),
    FOREIGN KEY (id_dgt_type) REFERENCES dbo.dim_dgt_type(id_dgt_type),
    FOREIGN KEY (id_municipality) REFERENCES dbo.dim_municipality(id_municipality)
);

```

Afegim un nou camp *id_municipality* que serà la forana que referencia la dimensió *dim_municipality*.

Al script del *Table output* afegim un nou *JOIN* per unir la taula *dim_municipality* i com a dada seleccionem també el *id_municipality*.

Table input

Step name: Table input (stg_vehicles)

Connection: cn_stg

SQL:

```

SELECT
    v.vehicle_id,
    dgt.id_dgt_type,
    s.registration_date,
    m.id_municipality,
    s.new_used,
    1 quantity
FROM dbo.stg_vehicles s
JOIN dbo.dim_vehicle v ON
    (s.brand = v.brand AND s.model = v.model AND s.subtype = v.subtype)
JOIN dbo.dim_dgt_type dgt ON
    s.subtype = dgt.cod_subtype_dgt
JOIN dbo.dim_municipality m ON
    s.municipality = m.cod_geo;

```

Line 1 Column 0

Store column info in step meta data ☐

Enable lazy conversion ☐

Replace variables in script? ☐

Insert data from step

Execute for each row? ☐

Limit size: 0

Help OK Preview Cancel

Comprovem que els canvis es reflecteixen també al *Table output*.

Table output

Step name: Table output

Connection: cn_stg

Target schema: dbo

Target table: fact_vehicle_registration

Commit size: 1000

Truncate table: ☒

Ignore insert errors: ☐

Specify database fields: ☒

Main options Database fields

Fields to insert:

#	Table field	Stream field
1	vehicle_id	vehicle_id
2	id_dgt_type	id_dgt_type
3	registration...	registration...
4	id_municipal...	id_municipal...
5	new_used	new_used
6	quantity	quantity
7	id_registrati...	pk

Get fields Enter field mapping

Help OK Cancel SQL

Finalment, executem i comprovem la correcte inserció de les dades a la taula.

Diagrama de flux: Table input (stg_vehicles) → Add sequence → Table output

Execution Results

Logging | Execution History | Step Metrics | Performance Graph | Metrics | Preview data

2025/12/16 13:37:49 - Table input (stg_vehicles).0 - !TableInput.Log.LineNumber!
2025/12/16 13:37:50 - Add sequence.0 - !linenr 1550000
2025/12/16 13:37:51 - Table output.0 - !linenr 1550000
2025/12/16 13:37:53 - Table input (stg_vehicles).0 - !TableInput.Log.LineNumber!
2025/12/16 13:37:54 - Add sequence.0 - !linenr 1600000
2025/12/16 13:37:55 - Table output.0 - !linenr 1600000
2025/12/16 13:37:57 - Table input (stg_vehicles).0 - !TableInput.Log.LineNumber!
2025/12/16 13:37:58 - Add sequence.0 - !linenr 1650000
2025/12/16 13:37:59 - Table output.0 - !linenr 1650000
2025/12/16 13:38:01 - Table input (stg_vehicles).0 - !TableInput.Log.LineNumber!
2025/12/16 13:38:02 - Add sequence.0 - !linenr 1700000
2025/12/16 13:38:02 - Table output.0 - !linenr 1700000
2025/12/16 13:38:02 - Table input (stg_vehicles).0 - !TableInput.Log.FinishedReadingQuery!
2025/12/16 13:38:02 - Table input (stg_vehicles).0 - Finished processing (I=1721704, O=0, R=0, W=1721704, U=0, E=0)
2025/12/16 13:38:03 - Add sequence.0 - Finished processing (I=0, O=0, R=1721704, W=1721704, U=0, E=0)
2025/12/16 13:38:04 - Table output.0 - Finished processing (I=0, O=1721704, R=1721704, W=1721704, U=0, E=0)
2025/12/16 13:38:04 - Spoon - The transformation has finished!!

Data Output

Messages

Notifications

	id_registration [PK] integer	registration_date date	vehicle_id integer	id_dgt_type integer	id_municipality integer	new_used boolean	quantity integer
1	1	2020-03-01	769	64	6969	true	1
2	2	2020-03-01	769	64	6969	true	1
3	3	2020-03-01	769	64	6969	true	1
4	4	2020-03-01	769	64	6969	true	1
5	5	2020-03-01	769	64	6969	true	1
6	6	2020-03-01	769	64	6969	true	1
7	7	2020-03-01	769	64	6969	true	1
8	8	2020-03-01	769	64	6969	true	1
9	9	2020-03-01	769	64	6969	true	1
10	10	2020-03-01	769	64	6969	true	1
11	11	2020-03-01	769	64	6969	true	1
12	12	2020-03-01	769	64	6969	true	1
13	13	2020-03-01	769	64	6969	true	1
14	14	2020-03-01	769	64	6969	true	1
15	15	2020-03-01	769	64	6969	true	1
16	16	2020-03-01	769	64	6969	true	1

Total rows: 1000 of 1721704

Query complete 00:00:03.910

Finalment, cal adaptar el *JOB_LOAD* per incorporar la transformació *DIM_MUNICIPALITY* i la modificació de *FACT_VEHICLE_REGISTRATION*.

Primer modifiquem l'script SQL de l'inici del job per eliminar els registres de les taules, incloent la taula *dim_municipality*.

SQL

Job entry name: SQL

Connection: cn_stg [Edit...] [New...] [Wizard...]

SQL from file: ☐

SQL filename: [Browse...]

Send SQL as single: ☐

Use variable substitution?: ☐

SQL Script:

```
TRUNCATE TABLE dbo.fact_vehicle_registration;  
DELETE FROM dbo.DIM_DGT_TYPE;  
DELETE FROM dbo.DIM_DATE;  
DELETE FROM dbo.DIM_VEHICLE;  
DELETE FROM dbo.DIM_MUNICIPALITY;
```

Line 5 Column 33

[Help] [OK] [Cancel]

Afegim la transformació *DIM_MUNICIPALITY*.

Transformation

Entry Name:

Transformation:

Options | Logging | Arguments | Parameters

Run configuration:

Execution

- ☐ Execute every input row
- ☐ Clear results rows before execution
- ☐ Clear results files before execution
- ☒ Wait for remote transformation to complete
- ☐ Follow local abort to remote transformation
- ☐ Suppress result data from remote transformation

Executem el job i comprovem la seva correcta execució i inserció de les dades.



Execution Results

Logging

History

Job metrics

Metrics

2025/12/16 13:51:49 - Add sequence.0 - linerr 1700000

2025/12/16 13:51:50 - Table output.0 - linerr 1700000

2025/12/16 13:51:50 - Table input (stg_vehicles).0 - !TableInput.Log.FinishedReadingQuery!

2025/12/16 13:51:50 - Table input (stg_vehicles).0 - Finished processing (l=1721704, O=0, R=0, W=1721704, U=0, E=0)

2025/12/16 13:51:51 - Add sequence.0 - Finished processing (l=0, O=0, R=1721704, W=1721704, U=0, E=0)

2025/12/16 13:51:52 - Table output.0 - Finished processing (l=0, O=1721704, R=1721704, W=1721704, U=0, E=0)

2025/12/16 13:51:52 - JOB_LOAD - Starting entry (Success)

2025/12/16 13:51:52 - JOB_LOAD - Finished job entry (Success) (result=[true])

2025/12/16 13:51:52 - JOB_LOAD - Finished job entry (FACT_VEHICLE_REGISTRATION) (result=[true])

2025/12/16 13:51:52 - JOB_LOAD - Finished job entry (DIM_MUNICIPALITY) (result=[true])

2025/12/16 13:51:52 - JOB_LOAD - Finished job entry (DIM_VEHICLE) (result=[true])

2025/12/16 13:51:52 - JOB_LOAD - Finished job entry (DIM_DATE) (result=[true])

2025/12/16 13:51:52 - JOB_LOAD - Finished job entry (DIM_DGT_TYPE) (result=[true])

2025/12/16 13:51:52 - JOB_LOAD - Finished job entry (SQL) (result=[true])

2025/12/16 13:51:52 - JOB_LOAD - Finished job entry (Set variables) (result=[true])

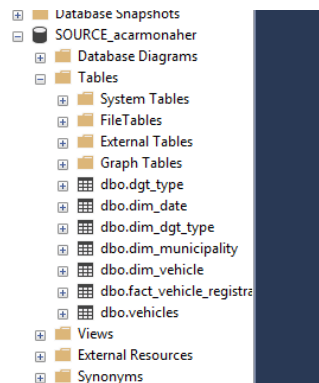
2025/12/16 13:51:52 - JOB_LOAD - Job execution finished

2025/12/16 13:51:52 - Spoon - Job has ended.

[illegible]

Trasllat al *data mart* instal·lat a la base de dades *SQL Server*

En primer lloc creem les taules a la base de dades, utilitzant l'script proporcionat a l'enunciat. Una vegada executat comprovem la seva correcta creació.

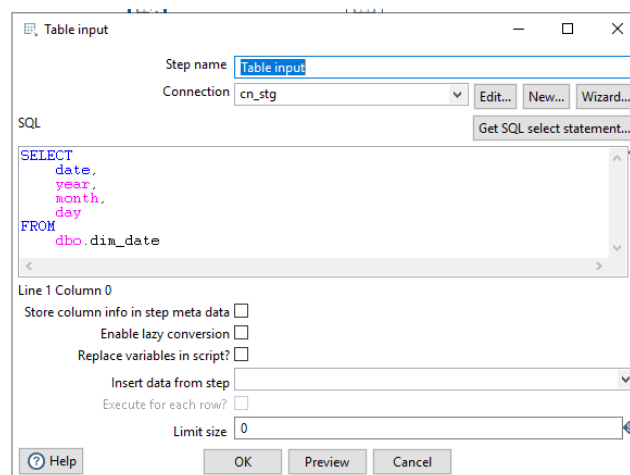


Per traslladar les dades a aquestes taules, creem una transformació per cada taula que traslladem: *DM_DIM_DATE*, *DM_DIM_VEHICLE*, *DM_DIM_MUNICIPALITY*, *DM_DIM_DGT_TYPE*, *DM_FACT_VEHICLE_REGISTRATION*.

Per cada una de les transformacions utilitzem un *Table input* connectat a la base de dades *PostgreSQL*, d'on recullim les dades; i un *Table output* connectat a la base de dades *PostgreSQL* per enviar les dades.

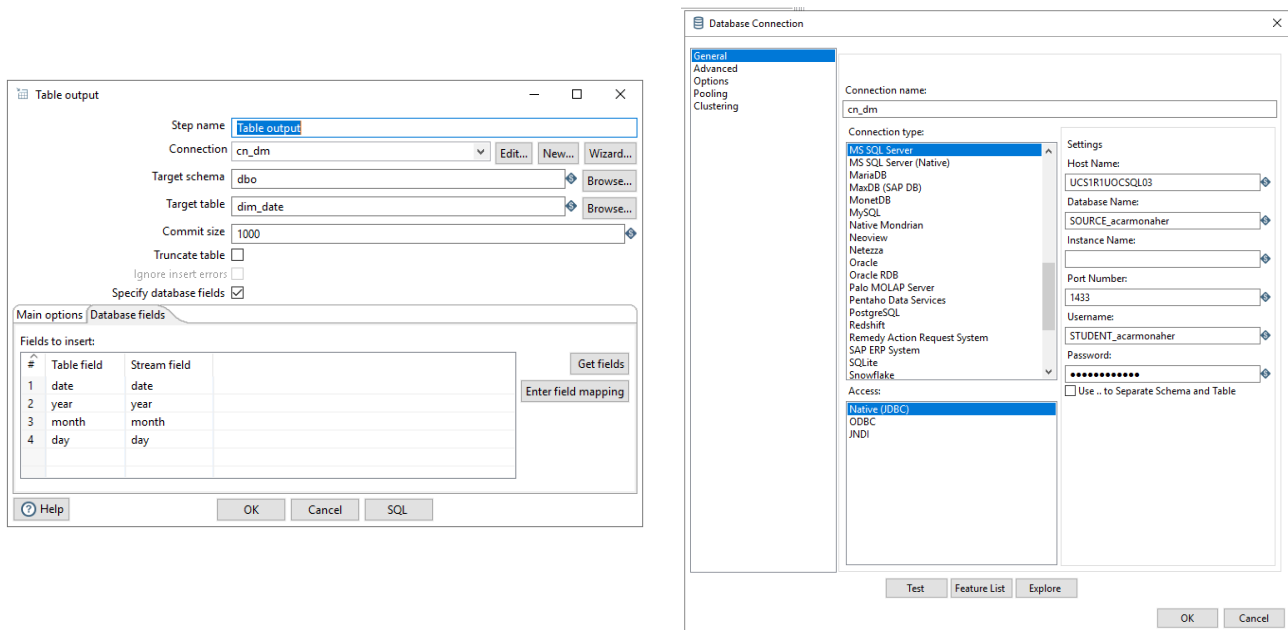
▪ Transformació *DM_DIM_DATE*

Creem el *Table input* de la següent manera:



Connectat a la base de dades *PostgreSQL* de la mateixa manera que havíem fet fins ara, i seleccionant totes les columnes de la taula *dim_date*.

Connectem el *Table input* a un *Table output*, que ha de ser de la següent manera:



Executem i comprovem la correcta inserció de les dades a la taula.

Execution Results

2025/12/16 14:09:25 - Spoon - Running transformation using the Kettle execution engine
 2025/12/16 14:09:25 - Spoon - Transformation opened.
 2025/12/16 14:09:25 - Spoon - Launching transformation [DM_DIM_DATE]...
 2025/12/16 14:09:25 - Spoon - Started the transformation execution.
 2025/12/16 14:09:25 - DM_DIM_DATE - Dispatching started for transformation [DM_DIM_DATE]
 2025/12/16 14:09:25 - Table output.0 - Connected to database [cn_dm] (commit=1000)
 2025/12/16 14:09:25 - Table input.0 - TableInput.Log.FinishedReadingQuery!
 2025/12/16 14:09:25 - Table input.0 - Finished processing (I=2177, O=0, R=0, W=2177, U=0, E=0)
 2025/12/16 14:09:25 - Table output.0 - Finished processing (I=0, O=2177, R=2177, W=2177, U=0, E=0)
 2025/12/16 14:09:25 - Spoon - The transformation has finished!!

	date	year	month	day
1	2020-01-01	2020	1	1
2	2020-01-02	2020	1	2
3	2020-01-03	2020	1	3
4	2020-01-04	2020	1	4
5	2020-01-05	2020	1	5
6	2020-01-06	2020	1	6
7	2020-01-07	2020	1	7
8	2020-01-08	2020	1	8
9	2020-01-09	2020	1	9
10	2020-01-10	2020	1	10
11	2020-01-11	2020	1	11
12	2020-01-12	2020	1	12
13	2020-01-13	2020	1	13
14	2020-01-14	2020	1	14
15	2020-01-15	2020	1	15
16	2020-01-16	2020	1	16
17	2020-01-17	2020	1	17
18	2020-01-18	2020	1	18
19	2020-01-19	2020	1	19
20	2020-01-20	2020	1	20

Query executed successfully.

- Transformació *DM_DIM_VEHICLE*

Repetim exactament el mateix procediment que per la transformació anterior, en aquest cas, per transferir les dades de la taula *dim_vehicle*.

Table input:

Table input

Step name: Table input

Connection: cn_stg

SQL:

```
SELECT
    vehicle_id,
    brand,
    model,
    subtype
FROM
    dbo.dim_vehicle;
```

Line 1 Column 0

Store column info in step meta data: ☐

Enable lazy conversion: ☐

Replace variables in script?: ☐

Insert data from step:

Execute for each row?: ☐

Limit size: 0

Help OK Preview Cancel

Table output:

Table output

Step name: Table output

Connection: cn_dm

Target schema: dbo

Target table: dim_vehicle

Commit size: 1000

Truncate table: ☐

Ignore insert errors: ☐

Specify database fields: ☒

Main options Database fields

Fields to insert:

#	Table field	Stream field
1	vehicle_id	vehicle_id
2	brand	brand
3	model	model
4	subtype	subtype

Get fields

Enter field mapping

Help OK Cancel SQL

Cal veure que la connexió en el *Table output* és altra vegada la de la base de dades *SQL Server*, com en la transformació anterior.

Executem i comprovem la correcta inserció de les dades.

Diagram showing the data flow from *Table input* to *Table output*.

Execution Results

Logging Execution History Step Metrics Performance Graph Metrics Preview d

2025/12/16 14:13:57 - Spoon - Running transformation using the Kettle execution engine

2025/12/16 14:13:57 - Spoon - Transformation opened.

2025/12/16 14:13:57 - Spoon - Launching transformation [DM_DIM_VEHICLE]...

2025/12/16 14:13:57 - Spoon - Started the transformation execution.

2025/12/16 14:13:57 - DM_DIM_VEHICLE - Dispatching started for transformation [DM_DIM_VEHICLE]

2025/12/16 14:13:57 - Table output.0 - Connected to database [cn_dm] (commit=1000)

2025/12/16 14:13:57 - Table input.0 - !TableInput.Log.FinishedReadingQuery!

2025/12/16 14:13:57 - Table input.0 - Finished processing (I=0, O=0, R=0, W=1257, U=0, E=0)

2025/12/16 14:13:57 - Table output.0 - Finished processing (I=0, O=1257, R=1257, W=1257, U=0, E=0)

2025/12/16 14:13:57 - Spoon - The transformation has finished!!!

Results Messages

vehicle_id	brand	model	subtype
1	BMW	218D	40
2	AIXAM	S9	92
3	YAMAHA	MTN690-U (MT-07)	50
4	CITROEN	JUMPER BLUEHDI 140 S&S	7A
5	VOLKSWAGEN	ID.4 PRO 150 KW	40
6	NISSAN	NV250	20
7	PEUGEOT	BOXER FURGÓN 335 L2 H2	20
8	ALFA ROMEO	GIULIA	40
9	PIAGGIO	MP3 300 HPE	53
10	CITROEN	C4 BLUEHDI 100 FEEL ED	40
11	PIAGGIO	LIBERTY	90
12	SMART	EQ FORTWO COUPE	40
13	BERGADANA	SPRINTER 910 AMBULANCI	22
14	MERCEDES-BENZ	GLC 250 4MATIC	40
15	CARROCERA CASTROSUA SA	MAGNUS E+ KIB4X2	30
16	PEUGEOT	NUEVO 308 5P ALLURE PA	40
17	FORD	TOURNEO COURIER	0G
18	RAY ELECTRIC MOTORS	RAY 7.7	51
19	AUDI	Q2	60
20	RENAULT	PR125 PR125S PR125S T	50

Query executed successfully.

- Transformació *DM_DIM_MUNICIPALITY*

Tornem a repetir el mateix procediment per inserir les dades a la taula *dim_municipality* de *SQL Server*.

Table input:

Step name: Table input

Connection: cn_stg

SQL:

```
SELECT
    id_municipality,
    cod_ine,
    cod_geo,
    cod_prov,
    provincia,
    name_muni,
    population_muni,
    surface,
    perimeter,
    cod_ine_capital,
    capital,
    population_capital,
    longitude_etr89,
    latitude_etr89,
    altitude
FROM
    dbo.dim_municipality;
```

Line 1 Column 0

Store column info in step meta data: ☐

Enable lazy conversion: ☐

Replace variables in script?: ☐

Insert data from step:

Execute for each row?: ☐

Limit size: 0

Buttons: Help, OK, Preview, Cancel

Table output:

Step name: Table output

Connection: cn_dm

Target schema: dbo

Target table: dim_municipality

Commit size: 1000

Truncate table: ☐

Ignore insert errors: ☐

Specify database fields: ☒

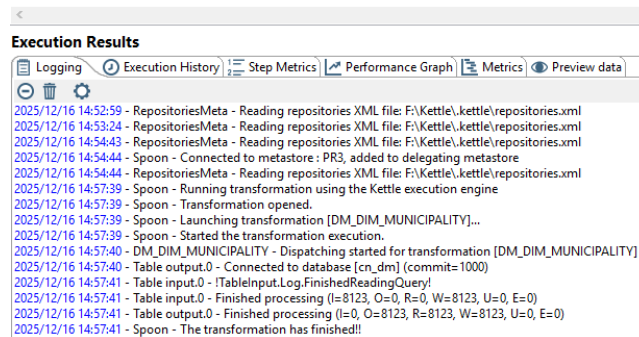
Main options | Database fields

Fields to insert:

#	Table field	Stream field
1	id_municip...	id_municipal...
2	cod_ine	cod_ine
3	cod_geo	cod_geo
4	cod_prov	cod_prov
5	provincia	provincia
6	name_muni	name_muni
7	population...	population_...
8	surface	surface
9	perimeter	perimeter
10	cod_ine_ca...	cod_ine_capi...
11	capital	capital
12	population...	population_C...
13	longitude...	longitude_etr...
14	latitude_etr...	latitude_etr89
15	altitude	altitude

Buttons: Help, OK, Cancel, SQL

Una vegada creats aquests dos passos, amb les connexions correctes en cada cas, executem i comprovem la correcta inserció de les dades a la taula.



	id_municipality	cod_jne	cod_geo	cod_prov	provincia	name_muni	population_muni	surface	perimeter	cod_jne_capital	capital	population_capital	longitude
1	1	09907000000	09023	09	Burgos	Alfoz de Quintanadueñas	2180	4150.8107	41552	09907000401	Quintanadueñas	1544	-3.73601
2	2	45146000000	45580	45	Toledo	Retamoso de la Jara	109	4824.4925	31818	45146000101	Retamoso de la Jara	109	-4.75501
3	3	47039000000	47160	47	Valladolid	Castriño-Tejeriego	157	3592.2368	30088	47039000101	Castriño-Tejeriego	157	-4.37071
4	4	47023000000	47096	47	Valladolid	Boecillo	4297	2411.8628	25099	47023000103	Boecillo	2146	-4.69931
5	5	07057000000	07560	07	Illes Balears	Santanyi	12480	12486.3057	86642	07057000801	Santanyi	3356	3.12944
6	6	48097000000	48950	48	Bizkaia	Zaratamo	1610	1018.3	13350	48097000502	Zaratamo	344	-2.87341
7	7	28026000000	28130	28	Madrid	Brunete	11014	4894.3516	32787	28026000101	Brunete	9631	-3.99851
8	8	19252000000	19657	19	Guadalajara	Sayatón	63	4537.5369	41534	19252000201	Sayatón	62	-2.85271
9	9	09446000000	09814	09	Burgos	Villamedianilla	9	675.825	13456	09446000101	Villamedianilla	9	-4.14551
10	10	14008000000	14080	14	Córdoba	Belalcázar	3138	35600.27	116200	14008000101	Belalcázar	3017	-5.16631
11	11	43106000000	43525	43	Tarragona	El Pinell de Brai	1006	5728.87	36148	43106000101	El Pinell de Brai	983	0.51511
12	12	49203000000	49567	49	Zamora	Santa Maria de la Vega	293	1788.1745	18872	49203000101	Santa Maria de la Vega	274	-5.80991
13	13	31155000000	31465	31	Navarra	Liédena	323	1927.981	21698	31155000101	Liédena	321	-1.27551
14	14	29074000000	29730	29	Málaga	Montejaque	970	4545.531	32658	29074000101	Montejaque	939	-5.25171
15	15	40206000000	40736	40	Segovia	Torre Val de San Pedro	190	4440.54	48746	40206000201	Torre Val de San Pedro	78	-3.86921
16	16	37302000000	37562	37	Salamanca	Saucelle	254	4608.73	32367	37302000201	Saucelle	254	-6.75131
17	17	22142000000	22476	22	Huesca	Lascuane	138	3187.0796	33651	22142000101	Lascuane	122	0.51593
18	18	15023000000	15230	15	A Coruña	Cee	7660	5745.3229	51596	15023000101	Cee	3992	-9.19021
19	19	42121000000	42520	42	Soria	Montenegro de Cameros	52	5532.1621	31275	42121000101	Montenegro de Cameros	52	-2.75381
20	20	19064000000	19169	19	Guadalajara	Comendador	87	6363.437	42170	19064000101	Comendador	84	-2.48991

Transformació *DM_DIM_DGT_TYPE*

Repetim el mateix procediment per transferir les dades de la taula *dim_dgt_type* de *PostgreSQL* a *SQL Server*.

Table input:

Table input

Step name

Table input

Connection

cn_stg

Edit...

New...

Wizard...

SQL

SELECT

id_dgt_type,

cod_type_dgt,

des_type_dgt,

cod_subtype_dgt,

des_subtype_dgt

FROM

dbo.dim_dgt_type;

Line 1 Column 0

Store column info in step meta data

Enable lazy conversion

Replace variables in script?

Insert data from step

Execute for each row?

Limit size

0

Help

OK

Preview

Cancel

Table output:

Step name: Table output

Connection: dm_cn

Target schema: dbo

Target table: dim_dgt_type

Commit size: 1000

Truncate table: ☐

Ignore insert errors: ☐

Specify database fields: ☒

Main options: Database fields

Fields to insert:

#	Table field	Stream field
1	id_dgt_type	id_dgt_type
2	cod_type_dgt	cod_type_dgt
3	des_type_dgt	des_type_dgt
4	cod_subtyp...	cod_subtype...
5	des_subtyp...	des_subtype...

Buttons: Get fields, Enter field mapping, Help, OK, Cancel, SQL

Executem i comprovem la correcta inserció de les dades a la taula *dim_dgt_type* de *SQL Server*.

	id_dgt_type	cod_type_dgt	des_type_dgt	cod_subtype_dgt	des_subtype_dgt
1	1	9	R Y S	SC	SEMIREMOLQUE HORMIGONERA
2	2	9	R Y S	R3	REMOLQUE FURGÓN
3	3	9	CICLOMOTORES	90	CICLOMOTOR DE 2 RUEDAS
4	4	9	R Y S	S9	SEMIREMOLQUE CANTERA
5	5	5	MOTOCICLETAS	51	MOTOCICLETA CON SIDECAR
6	6	9	R Y S	RE	REMOLQUE DE GRÚA
7	7	8	TRACTORES INDUSTRIALES	81	TRACTOCAMIÓN
8	8	1	CAMIONES	06	CAMIÓN JAULA
9	9	9	R Y S	SB	SEMIREMOLQUE VIAJEROS O AUTOBÚS
10	10	0	OTROS vehiculos	7J	PALA CARGADORA-RETROEXCAVADORA
11	11	5	MOTOCICLETAS	60	VEHÍCULO DE MOVILIDAD PERSONAL
12	12	5	MOTOCICLETAS	54	CUATRICICLO PESADO
13	13	9	R Y S	SF	SEMIREMOLQUE CONTRA INCENDIOS
14	14	3	AUTOBUSES	32	AUTOBÚS MIXTO
15	15	1	CAMIONES	00	CAMIÓN
16	16	0	OTROS vehiculos	77	GIROGRAVILLADORA
17	17	5	MOTOCICLETAS	53	AUTOMÓVIL DE 3 RUEDAS
18	18	9	R Y S	RD	REMOLQUE VOLQUETE DE CANTERA
19	19	1	CAMIONES	01	CAMIÓN PLATAFORMA
20	20	9	R Y S	RE	REMOLQUE TALLER

Query executed successfully.

▪ Transformació *DM_FACT_VEHICLE_REGISTRATION*

Finalment, creem la transformació *DM_FACT_VEHICLE_REGISTRATION* per transferir les dades de la taula *fact_vehicle_registration*, seguin els mateixos passos que pels casos anteriors.

Table input:

Table input

Step name

Table input

Connection

cn_stg

Edit...

New...

Wizard...

SQL

Get SQL select statement...

SELECT

id_registration,

registration_date,

vehicle_id,

id_dgt_type,

id_municipality,

new_used,

quantity

FROM

dbo.fact_vehicle_registration;

Line 1 Column 0

Store column info in step meta data

Enable lazy conversion

Replace variables in script?

Insert data from step

Execute for each row?

Limit size

Help

OK

Preview

Cancel

Tbae output:

Table output

Step name

Table output

Connection

cn_dm

Edit...

New...

Wizard...

Target schema

dbo

Browse...

Target table

fact_vehicle_registration

Browse...

Commit size

1000

Truncate table

Ignore insert errors

Specify database fields

Main options / Database fields

Fields to insert:

#	Table field	Stream field
1	id_registrati...	id_registration
2	date	registration_...
3	vehicle_id	vehicle_id
4	id_dgt_type	id_dgt_type
5	id_municip...	id_municipal...
6	new_used	new_used
7	quantity	quantity

Get fields

Enter field mapping

Help

OK

Cancel

SQL

Executem i comprovem la correcta inserció de les dades a la taula *fact_vehicle_registration*.

Execution Results

Logging Execution History Step Metrics Performance Graph Metrics Preview data

2025/12/16 15:09:32 - Table output.0 - linenr 1400000
2025/12/16 15:09:33 - Table input.0 - !TableInput.Log.LineNumber!
2025/12/16 15:09:34 - Table output.0 - linenr 1450000
2025/12/16 15:09:35 - Table input.0 - !TableInput.Log.LineNumber!
2025/12/16 15:09:35 - Table output.0 - linenr 1500000
2025/12/16 15:09:36 - Table input.0 - !TableInput.Log.LineNumber!
2025/12/16 15:09:37 - Table output.0 - linenr 1550000
2025/12/16 15:09:38 - Table input.0 - !TableInput.Log.LineNumber!
2025/12/16 15:09:38 - Table output.0 - linenr 1600000
2025/12/16 15:09:40 - Table input.0 - !TableInput.Log.LineNumber!
2025/12/16 15:09:40 - Table output.0 - linenr 1650000
2025/12/16 15:09:41 - Table input.0 - !TableInput.Log.LineNumber!
2025/12/16 15:09:41 - Table output.0 - linenr 1700000
2025/12/16 15:09:42 - Table input.0 - !TableInput.Log.FinishedReadingQuery!
2025/12/16 15:09:42 - Table input.0 - Finished processing (I=1721704, O=0, R=0, W=1721704, U=0, E=0)
2025/12/16 15:09:42 - Table output.0 - Finished processing (I=0, O=1721704, R=1721704, W=1721704, U=0, E=0)
2025/12/16 15:09:42 - Spoon - The transformation has finished!!!

Results

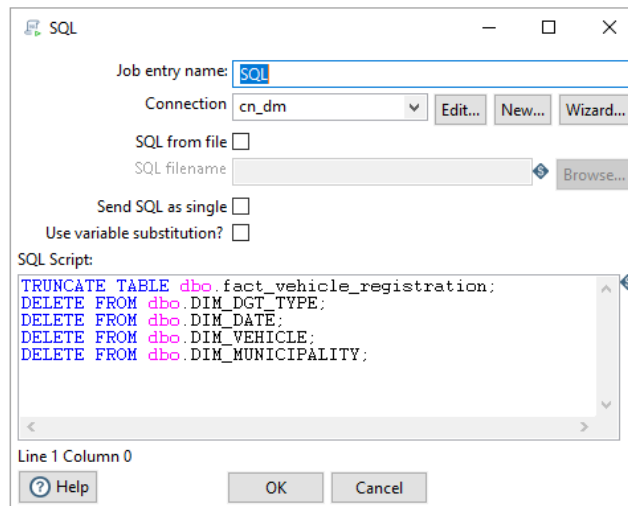
	id_registration	date	vehicle_id	new_used	id_dgt_type	id_municipality	quantity
1	1	2020-03-01	769	1	64	6969	1
2	2	2020-03-01	769	1	64	6969	1
3	3	2020-03-01	769	1	64	6969	1
4	4	2020-03-01	769	1	64	6969	1
5	5	2020-03-01	769	1	64	6969	1
6	6	2020-03-01	769	1	64	6969	1
7	7	2020-03-01	769	1	64	6969	1
8	8	2020-03-01	769	1	64	6969	1
9	9	2020-03-01	769	1	64	6969	1
10	10	2020-03-01	769	1	64	6969	1
11	11	2020-03-01	769	1	64	6969	1
12	12	2020-03-01	769	1	64	6969	1
13	13	2020-03-01	769	1	64	6969	1
14	14	2020-03-01	769	1	64	6969	1
15	15	2020-03-01	769	1	64	6969	1
16	16	2020-03-01	769	1	64	6969	1
17	17	2020-03-01	769	1	64	6969	1
18	18	2020-03-01	769	1	64	6969	1
19	19	2020-03-01	769	1	64	6969	1

Job *JOB_DM_LOAD*

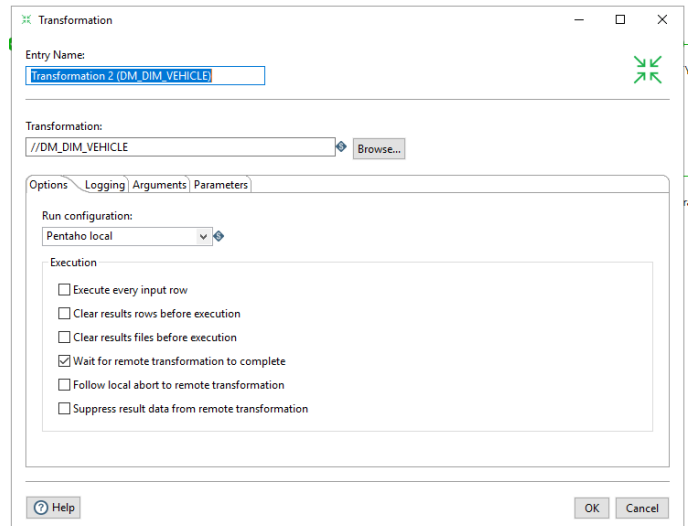
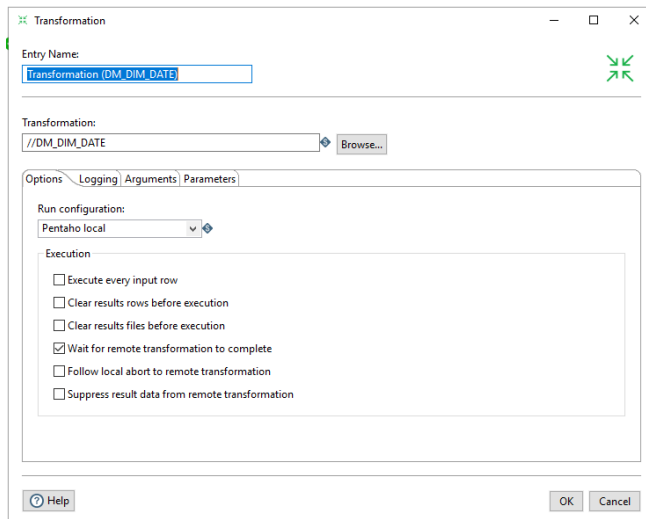
Finalment, creem un job anomenat *JOB_DM_LOAD*, que integri l'execució de la còpia del *data warehouse* al *data mart*.

El job ha d'incloure un script *SQL* i les transformacions *DM*, similar al cas del *JOB_LOAD*.

L'script *SQL* és el mateix, que serveix per eliminar els registres de les taules:



Introduïm cada una de les transformacions.



Transformation

Entry Name: Transformation 3 (DM_DIM_MUNICIPALITY)

Transformation: //DM_DIM_MUNICIPALITY

Options | Logging | Arguments | Parameters

Run configuration: Pentaho local

Execution

- ☐ Execute every input row
- ☐ Clear results rows before execution
- ☐ Clear results files before execution
- ☒ Wait for remote transformation to complete
- ☐ Follow local abort to remote transformation
- ☐ Suppress result data from remote transformation

Help OK Cancel

Transformation

Entry Name: Transformation 4 (DM_DIM_DGT_TYPE)

Transformation: //DM_DIM_DGT_TYPE

Options | Logging | Arguments | Parameters

Run configuration: Pentaho local

Execution

- ☐ Execute every input row
- ☐ Clear results rows before execution
- ☐ Clear results files before execution
- ☒ Wait for remote transformation to complete
- ☐ Follow local abort to remote transformation
- ☐ Suppress result data from remote transformation

Help OK Cancel

Transformation

Entry Name: Transformation 5 (DM_FACT_VEHICLE_REGISTRATION)

Transformation: //DM_FACT_VEHICLE_REGISTRATION

Options | Logging | Arguments | Parameters

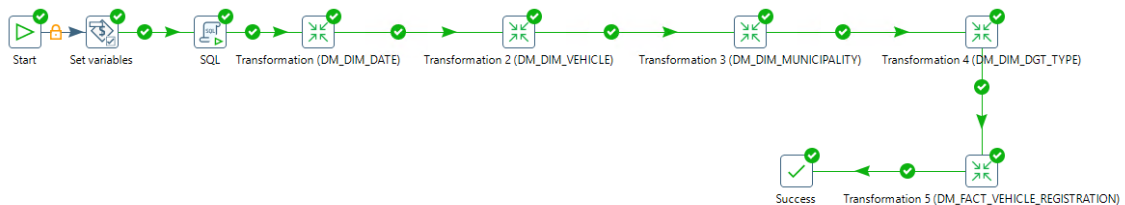
Run configuration: Pentaho local

Execution

- ☐ Execute every input row
- ☐ Clear results rows before execution
- ☐ Clear results files before execution
- ☒ Wait for remote transformation to complete
- ☐ Follow local abort to remote transformation
- ☐ Suppress result data from remote transformation

Help OK Cancel

Executem i comprovem el correcte funcionament del job.



Execution Results

Logging | History | Job metrics | Metrics

2025/12/16 15:22:42 - Table output.0 - linetr 1650000

2025/12/16 15:22:44 - Table input.0 - !TableInput.Log.LineNumber!

2025/12/16 15:22:44 - Table output.0 - linetr 1700000

2025/12/16 15:22:44 - Table input.0 - !TableInput.Log.FinishedReadingQuery!

2025/12/16 15:22:44 - Table input.0 - Finished processing (I=1721704, O=0, R=0, W=1721704, U=0, E=0)

2025/12/16 15:22:45 - Table output.0 - Finished processing (I=0, O=1721704, R=1721704, W=1721704, U=0, E=0)

2025/12/16 15:22:45 - JOB_DM_LOAD - Starting entry [Success] (result=[true])

2025/12/16 15:22:45 - JOB_DM_LOAD - Finished job entry [Transformation 5 (DM_FACT_VEHICLE_REGISTRATION)] (result=[true])

2025/12/16 15:22:45 - JOB_DM_LOAD - Finished job entry [Transformation 4 (DM_DIM_DGT_TYPE)] (result=[true])

2025/12/16 15:22:45 - JOB_DM_LOAD - Finished job entry [Transformation 3 (DM_DIM_MUNICIPALITY)] (result=[true])

2025/12/16 15:22:45 - JOB_DM_LOAD - Finished job entry [Transformation 2 (DM_DIM_VEHCLE)] (result=[true])

2025/12/16 15:22:45 - JOB_DM_LOAD - Finished job entry [Transformation (DM_DIM_DATE)] (result=[true])

2025/12/16 15:22:45 - JOB_DM_LOAD - Finished job entry [SQL] (result=[true])

2025/12/16 15:22:45 - JOB_DM_LOAD - Finished job entry [Set variables] (result=[true])

2025/12/16 15:22:45 - JOB_DM_LOAD - Job execution finished

2025/12/16 15:22:45 - Spoon - Job has ended.

Activar
Ve a Config

100 %

Results Messages

	id_registration	date	vehicle_id	new_used	id_dgt_type	id_municipality	quantity
1	1	2020-03-01	769	1	64	6969	1
2	2	2020-03-01	769	1	64	6969	1
3	3	2020-03-01	769	1	64	6969	1
4	4	2020-03-01	769	1	64	6969	1
5	5	2020-03-01	769	1	64	6969	1
6	6	2020-03-01	769	1	64	6969	1
7	7	2020-03-01	769	1	64	6969	1
8	8	2020-03-01	769	1	64	6969	1
9	9	2020-03-01	769	1	64	6969	1
10	10	2020-03-01	769	1	64	6969	1
11	11	2020-03-01	769	1	64	6969	1
12	12	2020-03-01	769	1	64	6969	1
13	13	2020-03-01	769	1	64	6969	1
14	14	2020-03-01	769	1	64	6969	1
15	15	2020-03-01	769	1	64	6969	1
16	16	2020-03-01	769	1	64	6969	1
17	17	2020-03-01	769	1	64	6969	1
18	18	2020-03-01	769	1	64	6969	1
19	19	2020-03-01	769	1	64	6969	1
20	20	2020-03-01	769	1	64	6969	1

Query executed successfully.

Exercici 4. Ús de consultes Common Table Expressions (CTE) per analitzar les dades

1. Conèixer els deu primers (top ten) municipis amb més nombre de matriculacions a la província de Barcelona ordenats de forma descendent per nombre de matriculacions.

La consulta és la següent:

```
WITH mun_registrations AS(
    SELECT
        m.name_muni,
        m.cod_geo,
        SUM(r.quantity) AS total_reg
    FROM
        dbo.fact_vehicle_registration r
    JOIN
        dbo.dim_municipality m ON
        r.id_municipality = m.id_municipality
    WHERE
        m.provincia = 'Barcelona'
    GROUP BY
        m.name_muni,
        m.cod_geo
)
SELECT TOP 10
    mr.name_muni,
    mr.cod_geo,
    mr.total_reg
FROM
    mun_registrations mr
ORDER BY
    mr.total_reg DESC;
```

Aquesta consulta primer crea una CTE, *mun_registrations*, que pren la informació de la taula *fact_vehicle_registration*, amb un *JOIN* sobre la taula *dim_municipality*, per obtenir tots els registres de vehicles i la informació del municipi on han sigut registrats.

A partir d'aquí, agrupa els registres pel nom del municipi i el codi (*cod_geo*). No podem ordenar només pel nom ja que hi ha municipis que repeteixen nom. En canvi, sí que podríem agrupar únicament pel codi (tots són únics), però al agrupar pels dos d'aquesta manera, podem mostrar també el nom del municipi al resultat.

A més, filtra pels municipis de la província de Barcelona.

D'aquests grups, calcula el recompte sumant sobre la columna *quantity*.

Finalment, d'aquesta taula *mun_registrations*, selecciona el recompte (*total_reg*), el nom i el codi, els ordena de manera descendent i selecciona els 10 primers amb la clàusula *TOP 10*.

El resultat és el següent:

```
SQLQuery6.sql - UC....acarmonaher (83))*      SQLQuery5.sql - UC....acarmonaher (81))
WITH mun_registrations AS(
    SELECT
        m.name_muni,
        m.cod_geo,
        SUM(r.quantity) AS total_reg
    FROM
        dbo.fact_vehicle_registration r
    JOIN
        dbo.dim_municipality m ON
        r.id_municipality = m.id_municipality
    WHERE
        m.provincia = 'Barcelona'
    GROUP BY
        m.name_muni,
        m.cod_geo
)
SELECT TOP 10
    mr.name_muni,
    mr.cod_geo,
    mr.total_reg
FROM
    mun_registrations mr
ORDER BY
    mr.total_reg DESC;
```

	name_muni	cod_geo	total_reg
1	La Granada	08279	416993
2	L'Ametlla del Vallès	08015	371469
3	Campins	08114	127680
4	Caldes de Montbui	08096	102410
5	Calella	08102	59698
6	Font-rubí	08252	55803
7	Castellgalí	08180	55632
8	L'Espunyola	08231	46360
9	Cànoves i Samalús	08123	44973
10	Gelida	08270	43149

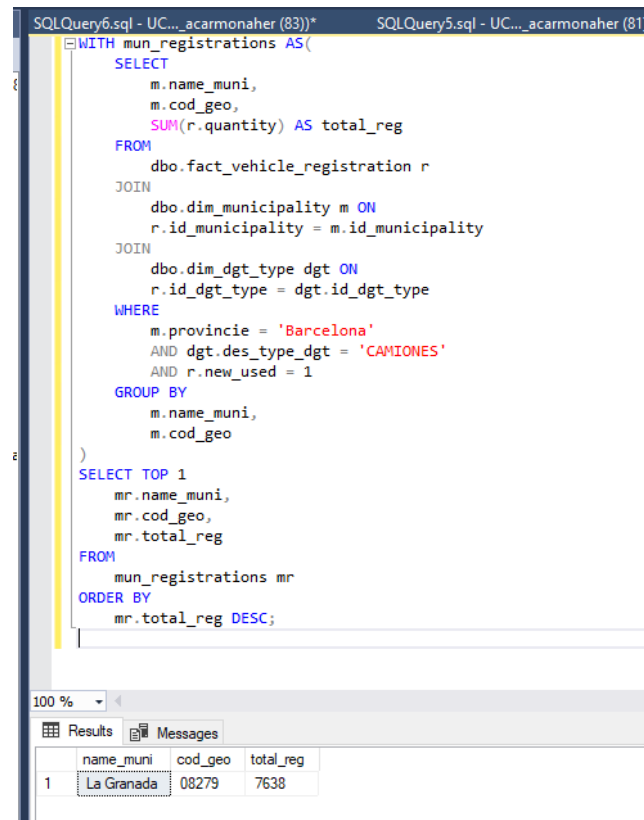
2. Repetir la consulta anterior per conèixer el municipi amb més nombre de matriculacions de “CAMIONES” nous.

En aquest cas, hem de prendre la consulta anterior, realitzar un altre *JOIN* per incloure la taula *dim_dgt_type*, filtrar pels que tenen *des_type_dgt* “CAMIONES”, filtrar pels que són nous (*new_used* = 1, ja que *SQL Server* utilitza 1 i 0 per representar camps de tipus *BIT*) i seleccionar el primer (*TOP 1*).

La consulta és la següent:

```
WITH mun_registrations AS(
    SELECT
        m.name_muni,
        m.cod_geo,
        SUM(r.quantity) AS total_reg
    FROM
        dbo.fact_vehicle_registration r
    JOIN
        dbo.dim_municipality m ON
        r.id_municipality = m.id_municipality
    JOIN
        dbo.dim_dgt_type dgt ON
        r.id_dgt_type = dgt.id_dgt_type
    WHERE
        m.provincia = 'Barcelona'
        AND dgt.des_type_dgt = 'CAMIONES'
        AND r.new_used = 1
    GROUP BY
        m.name_muni,
        m.cod_geo
)
SELECT TOP 1
    mr.name_muni,
    mr.cod_geo,
    mr.total_reg
FROM
    mun_registrations mr
ORDER BY
    mr.total_reg DESC;
```

Obtenim el següent resultat:



The screenshot shows a SQL query in the 'SQLQuery6.sql' window. The query uses a CTE named 'mun_registrations' to aggregate vehicle registration data by municipality and vehicle type. It filters for Barcelona province and trucks ('CAMIONES'). The final result is a table with one row for 'La Granada'.

```
SQLQuery6.sql - UC..._acarmonaher (83))*    SQLQuery5.sql - UC..._acarmonaher (81))
WITH mun_registrations AS(
    SELECT
        m.name_muni,
        m.cod_geo,
        SUM(r.quantity) AS total_reg
    FROM
        dbo.fact_vehicle_registration r
    JOIN
        dbo.dim_municipality m ON
        r.id_municipality = m.id_municipality
    JOIN
        dbo.dim_dgt_type dgt ON
        r.id_dgt_type = dgt.id_dgt_type
    WHERE
        m.provincia = 'Barcelona'
        AND dgt.des_type_dgt = 'CAMIONES'
        AND r.new_used = 1
    GROUP BY
        m.name_muni,
        m.cod_geo
)
SELECT TOP 1
    mr.name_muni,
    mr.cod_geo,
    mr.total_reg
FROM
    mun_registrations mr
ORDER BY
    mr.total_reg DESC;
```

100 %

Results Messages

	name_muni	cod_geo	total_reg
1	La Granada	08279	7638

3. Conèixer el total de matriculacions de vehicles de tipus “FURGONETAS” i de “TURISMOS” per cada municipi de la província de Barcelona, ordenat per nombre de matriculacions de “FURGONETAS” i nombre de matriculacions de “TURISMOS” de forma descendent.

En aquest cas, hem de fer un recompte de turismes i furgonetes matriculades per separat. Per fer-ho, fem una suma de la columna quantity amb *SUM*, però, per separar la suma dels tipus de vehicles, utilitzem la clàusula *CASE WHEN*, indicant que quan el tipus sigui el determinat, el valor a sumar és el de *quantity*, en cas contrari, és 0.

La consulta és la següent:

```
WITH mun_registrations AS(
    SELECT
        m.name_muni,
        m.cod_geo,
        SUM(CASE WHEN dgt.des_type_dgt = 'FURGONETAS' THEN r.quantity ELSE 0
END) AS total_furgonetas,
        SUM(CASE WHEN dgt.des_type_dgt = 'TURISMOS' THEN r.quantity ELSE 0 END)
AS total_turismos
    FROM
        dbo.fact_vehicle_registration r
    JOIN
        dbo.dim_municipality m ON
        r.id_municipality = m.id_municipality
    JOIN
        dbo.dim_dgt_type dgt ON
        r.id_dgt_type = dgt.id_dgt_type
    WHERE
        m.provincia = 'Barcelona'
        AND dgt.des_type_dgt in ('FURGONETAS', 'TURISMOS')
    GROUP BY
        m.name_muni,
        m.cod_geo
)
SELECT
    mr.name_muni,
    mr.cod_geo,
    mr.total_turismos,
    mr.total_furgonetas
FROM
    mun_registrations mr
ORDER BY
    mr.total_furgonetas DESC,
    mr.total_turismos DESC;
```

I obtenim el següent resultat:

SQLQuery6.sql - UC..._acarmonaher (83)* SQLQuery5.sql - UC..._acarmonaher (81) SQLQuery4.sql - UC..._acarmonaher (80)

```
WITH mun_registrations AS(
    SELECT
        m.name_muni,
        m.cod_geo,
        SUM(CASE WHEN dgt.des_type_dgt = 'FURGONETAS' THEN r.quantity ELSE 0 END) AS total_furgonetas,
        SUM(CASE WHEN dgt.des_type_dgt = 'TURISMOS' THEN r.quantity ELSE 0 END) AS total_turismos
    FROM
        dbo.fact_vehicle_registration r
    JOIN
        dbo.dim_municipality m ON
        r.id_municipality = m.id_municipality
    JOIN
        dbo.dim_dgt_type dgt ON
        r.id_dgt_type = dgt.id_dgt_type
    WHERE
        m.provincia = 'Barcelona'
        AND dgt.des_type_dgt in ('FURGONETAS', 'TURISMOS')
    GROUP BY
        m.name_muni,
        m.cod_geo
)
SELECT
    mr.name_muni,
    mr.cod_geo,
    mr.total_turismos,
    mr.total_furgonetas
FROM
    mun_registrations mr
ORDER BY
    mr.total_furgonetas DESC,
    mr.total_turismos DESC;
```

100 %

Results Messages

	name_muni	cod_geo	total_turismos	total_furgonetas
1	La Granada	08279	330239	19608
2	L'Ametlla del Vallès	08015	261212	14155
3	Caldes de Montbui	08096	86260	5624
4	Calella	08102	51186	3002
5	Font-rubí	08252	46265	2318
6	Castellgalí	08180	47937	1862
7	Balenyà	08051	35264	1672
8	Cànoves i Samalús	08123	37886	1653
9	Gelida	08270	30438	1634
10	Campins	08114	105773	1311
11	Cubelles	08219	26391	1159
12	Castellar del Vallès	08156	20995	1159
13	Castellbell i el Vilar	08159	28310	1045
14	L'Espunyola	08231	35948	1007
15	Balsareny	08054	21280	950

✓ Query executed successfully.

4. Des d'un punt de vista estadístic, es vol analitzar els municipis de la província de Barcelona per calcular en una única consulta:

- El nombre total de matriculacions per municipi.
- El nombre de matriculacions corresponents a vehicles de tipus “MOTOCICLETAS” per municipi.

A partir de les matriculacions de “MOTOCICLETAS” per municipi, es calcula:

- El percentatge que representen sobre el total de matriculacions del mateix municipi.
- El percentatge que representen sobre el total general de matriculacions de la província.

Els resultats s'han d'ordenar de forma descendent segons el nombre de matriculacions de “MOTOCICLETAS” per municipi.

La consulta és:

```
WITH total_barcelona AS(
    SELECT
        SUM(r.quantity) AS total_barcelona
    FROM
        dbo.fact_vehicle_registration r
    JOIN
        dbo.dim_municipality m ON
        r.id_municipality = m.id_municipality
    WHERE
        m.provincia = 'Barcelona'
),
total_muni AS(
    SELECT
        cod_geo,
        name_muni,
        SUM(r.quantity) AS total_muni
    FROM
        dbo.fact_vehicle_registration r
    JOIN
        dbo.dim_municipality m ON
        r.id_municipality = m.id_municipality
    WHERE
        m.provincia = 'Barcelona'
    GROUP BY
        cod_geo,
        name_muni
),
total_motocicletas AS(
    SELECT
        cod_geo,
        name_muni,
        SUM(r.quantity) AS total_moto
    FROM
        dbo.fact_vehicle_registration r
    JOIN
        dbo.dim_municipality m ON
        r.id_municipality = m.id_municipality
```

```

        JOIN
            dbo.dim_dgt_type dgt ON
            r.id_dgt_type = dgt.id_dgt_type
        WHERE
            m.provincia = 'Barcelona'
            AND dgt.des_type_dgt = 'MOTOCICLETAS'
        GROUP BY
            cod_geo,
            name_muni
    )

    SELECT
        tm.name_muni,
        tm.cod_geo,
        tb.total_barcelona,
        tm.total_moto,
        tmun.total_muni,
        ROUND(((100.0 * tm.total_moto) / tmun.total_muni), 3) AS muni_percentage,
        ROUND(((100.0 * tm.total_moto) / tb.total_barcelona), 3) AS bcn_percentage
    FROM total_muni tmun
    LEFT JOIN total_motocicletas tm ON
        tm.cod_geo = tmun.cod_geo
    CROSS JOIN total_barcelona tb
    ORDER BY
        total_moto DESC;

```

En aquest cas, he utilitzat 3 CTE. La primera, *total_barcelona*, calcula el total de registres a la província de Barcelona; la segona, *total_muni*, calcula el total de registres per municipi a la província de Barcelona; i la tercera, *total_motocicletas*, calcula el total de registres per municipi a la província de Barcelona i que siguin “MOTOCICLETAS”.

A partir d’aquí, es selecciona la taula *total_muni* i es fa un *LEFT JOIN* amb la taula *total_motocicletas* de manera que tenim els municipis amb el total de registres de motos i en total (i si un municipi no té registres de motos, també apareixerà). També es fa un *CROSS JOIN* amb *total_barcelona*, de manera que totes les files també tenen com a dada el total de registres de Barcelona.

A partir d’aquí, al *SELECT* es seleccionen els municipis, el total de Barcelona, el total de “MOTOCICLETAS”, el total per municipi i es calculen els percentatges de *total_moto/total_muni* i de *total_moto/total_barcelona*.

Finalment, s’ordena per *total_moto*.

Obtenim el següent resultat:

SQLQuery6.sql - UC..._acarmonaher (83))* SQLQuery5.sql - UC..._acarmonaher (81)) SQLQuery4.sql - UC..._acarmonaher

```
WITH total_barcelona AS(  
    SELECT  
        SUM(r.quantity) AS total_barcelona  
    FROM  
        dbo.fact_vehicle_registration r  
    JOIN  
        dbo.dim_municipality m ON  
        r.id_municipality = m.id_municipality  
    WHERE  
        m.provincia = 'Barcelona'  
),  
total_muni AS(  
    SELECT  
        cod_geo,  
        name_muni,  
        SUM(r.quantity) AS total_muni  
    FROM  
        dbo.fact_vehicle_registration r  
    JOIN  
        dbo.dim_municipality m ON  
        r.id_municipality = m.id_municipality  
    WHERE  
        m.provincia = 'Barcelona'  
    GROUP BY  
        cod_geo,  
        name_muni  
),  
total_motocicletas AS(  
    SELECT  
        cod_geo,
```

100 %

Results Messages

	name_muni	cod_geo	total_barcelona	total_moto	total_muni	muni_percentage	bcn_percentage
1	L'Ametlla del Vallès	08015	1721704	84683	371469	22.7970000000000	4.9190000000000
2	La Granada	08279	1721704	53713	416993	12.8810000000000	3.1200000000000
3	Campins	08114	1721704	18430	127680	14.4350000000000	1.0700000000000
4	Gelida	08270	1721704	10013	43149	23.2060000000000	0.5820000000000
5	L'Espunyola	08231	1721704	7429	46360	16.0250000000000	0.4310000000000
6	Caldes de Montbui	08096	1721704	5491	102410	5.3620000000000	0.3190000000000
7	Castellgalí	08180	1721704	4826	55632	8.6750000000000	0.2800000000000
8	Cànoves i Samalús	08123	1721704	4351	44973	9.6750000000000	0.2530000000000
9	Font-rubí	08252	1721704	3724	55803	6.6730000000000	0.2160000000000
10	Calella	08102	1721704	3173	59698	5.3150000000000	0.1840000000000
11	Cubelles	08219	1721704	2964	30951	9.5760000000000	0.1720000000000
12	Aguilar de Segarra	08006	1721704	2622	19456	13.4770000000000	0.1520000000000
13	Canyelles	08126	1721704	2394	18297	13.0840000000000	0.1390000000000
14	Balenyà	08051	1721704	2337	40394	5.7860000000000	0.1360000000000
15	Abrera	08003	1721704	1881	15770	11.9280000000000	0.1090000000000

Query executed successfully.

5. Analitzar des d'un punt de vista estadístic l'evolució de les matriculacions del subtipus de vehicle "TURISMO".

Per a això, en una única consulta, cal calcular:

- El total anual de matriculacions de la província.
- El total mensual de matriculacions, desglossat per any i mes.
- El total mensual de matriculacions del subtipus "TURISMOS", també per any i mes.

A partir d' aquests valors, calcular:

- El percentatge que representen les matriculacions mensuals de TURISMES sobre el total de matriculacions del mateix mes.
- El percentatge que representen les matriculacions mensuals de TURISMES sobre el total de matriculacions de l'any corresponent.

Els resultats s'han de presentar ordenats cronològicament per any i mes.

La consulta és la següent:

```
WITH total_per_year AS(
    SELECT
        d.year,
        SUM(r.quantity) AS total_year
    FROM
        dbo.fact_vehicle_registration r
    JOIN
        dbo.dim_date d ON
        r.date = d.date
    JOIN dbo.dim_municipality m ON
        r.id_municipality = m.id_municipality
    WHERE
        m.provincia = 'Barcelona'
    GROUP BY
        d.year
),
total_per_month AS(
    SELECT
        d.year,
        d.month,
        SUM(r.quantity) AS total_month
    FROM
        dbo.fact_vehicle_registration r
    JOIN
        dbo.dim_date d ON
        r.date = d.date
    JOIN dbo.dim_municipality m ON
        r.id_municipality = m.id_municipality
    WHERE
        m.provincia = 'Barcelona'
    GROUP BY
        d.year,
        d.month
),
```



```

total_month_turismo AS(
    SELECT
        d.year,
        d.month,
        SUM(r.quantity) AS total_month
    FROM
        dbo.fact_vehicle_registration r
    JOIN
        dbo.dim_date d ON
        r.date = d.date
    JOIN dbo.dim_municipality m ON
        r.id_municipality = m.id_municipality
    JOIN dbo.dim_dgt_type dgt ON
        r.id_dgt_type = dgt.id_dgt_type
    WHERE
        m.provincia = 'Barcelona'
        AND dgt.des_type_dgt = 'TURISMOS'
    GROUP BY
        d.year,
        d.month
)

SELECT
    ty.year,
    ty.total_year,
    tm.month,
    tm.total_month,
    tt.total_month AS total_turismos,
    ROUND(((100.0 * tt.total_month) / tm.total_month), 3) AS
turismo_percentage,
    ROUND(((100.0 * tt.total_month) / ty.total_year), 3) AS year_percentage
FROM total_per_year ty
JOIN total_per_month tm ON
    ty.year = tm.year
JOIN total_month_turismo tt ON
    ty.year = tt.year
    AND tm.month = tt.month
ORDER BY
    ty.year,
    tm.month;

```

En aquest cas s’han utilitzat 3 CTE. La primera, *total_per_year*, agrupa els registres per any i calcula el total de registres de cada any; la segona, *total_per_month*, que agrupa els registres per mes i any i calcula el total de registres de cada mes de cada any; i la tercera, *total_month_turismo*, que agrupa els registres per mes i any i filtra pel que son “TURISMOS”.

A partir d’aquí, selecciona la informació de la taula *total_per_year*, amb un *JOIN* sobre la taula *total_per_month*, i un altre *JOIN* sobre la taula *total_month_turismo*. D’aquí, selecciona l’any, el mes, el total de vehicles registrats per mes, el total de turismes registrats per mes i calcula els percentatges de turismes registrats del total del mes i del total de l’any.

Els resultats són els següents:

SQLQuery6.sql - UC...acarmonaher (83)) SQLQuery5.sql - UC...acarmonaher (81)) SQLQuery4.sql - UC...acarmonaher (59))

WITH total_per_year AS(
SELECT
d.year,
SUM(r.quantity) AS total_year
FROM
dbo.fact_vehicle_registration r
JOIN
dbo.dim_date d ON
r.date = d.date
JOIN dbo.dim_municipality m ON
r.id_municipality = m.id_municipality
WHERE
m.provincia = 'Barcelona'
GROUP BY
d.year
),

total_per_month AS(
SELECT
d.year,
d.month,
SUM(r.quantity) AS total_month
FROM
dbo.fact_vehicle_registration r
JOIN
dbo.dim_date d ON
r.date = d.date
JOIN dbo.dim_municipality m ON
r.id_municipality = m.id_municipality
)

100 %

Results Messages

	year	total_year	month	total_month	total_turismos	turismo_percentage	year_percentage
1	2020	450566	1	32832	25935	78.99300000000000	5.756000000000000
2	2020	450566	2	44517	35530	79.81200000000000	7.886000000000000
3	2020	450566	3	55442	45505	82.07700000000000	10.100000000000000
4	2020	450566	4	53200	45125	84.82100000000000	10.015000000000000
5	2020	450566	5	40033	32528	81.25300000000000	7.219000000000000
6	2020	450566	6	58311	48830	83.74100000000000	10.837000000000000
7	2020	450566	7	49077	41135	83.81700000000000	9.130000000000000
8	2020	450566	8	25099	16986	67.67600000000000	3.770000000000000
9	2020	450566	9	25213	17423	69.10300000000000	3.867000000000000
10	2020	450566	10	24054	19114	79.46300000000000	4.242000000000000
11	2020	450566	11	23408	19304	82.46800000000000	4.284000000000000
12	2020	450566	12	19380	15694	80.98000000000000	3.483000000000000
13	2021	396131	1	33193	26182	78.87800000000000	6.609000000000000
14	2021	396131	2	37297	29849	80.03100000000000	7.535000000000000
15	2021	396131	3	42047	35264	83.86800000000000	8.902000000000000
16	2021	396131	4	53485	40831	76.34100000000000	10.307000000000000
17	2021	396131	5	50407	41591	82.51000000000000	10.499000000000000

Query executed successfully.